UNIT -1

1. Demonstrate the significance of the data preparation phase of a datascience process with suitable examples.

What is data preparation?

Data preparation is the process of cleaning and transforming raw data prior to processing and analysis. It is an important step prior to processing and often involves reformatting data, making corrections to data, and combining datasets to enrich data.

Data preparation is often a lengthy undertaking for data engineers or business users, but it is essential as a prerequisite to put data in context in order to turn it into insights and eliminate bias resulting from poor data quality.

For example, the data preparation process usually includes standardizing data formats, enriching source data, and/or removing outliers.

Benefits of data preparation in the cloud

Data Scientist say that data preparation is the worst part of their job, but efficient, accurate business decisions can only be made with clean data. Data preparation helps:

- Fix errors quickly — Data preparation helps catch errors before processing. After data has been removed from its original source, these errors become more difficult to understand and correct.
- Produce top-quality data — Cleaning and reformatting datasets ensures that all data used in analysis will be of high quality.
- Make better business decisions — Higher-quality data that can be processed and analyzed more quickly and efficiently leads to more timely, efficient, better-quality business decisions.

Additionally, as data and data processes move to the cloud, data preparation moves with it for even greater benefits, such as:

- Superior scalability — Cloud data preparation can grow at the pace of the business. Enterprises don't have to worry about the underlying infrastructure or try to anticipate their evolutions.
- Future proof — Cloud data preparation upgrades automatically so that new capabilities or problem fixes can be turned on as soon as they are released. This allows organizations to stay ahead of the innovation curve without delays and added costs.
- Accelerated data usage and collaboration — Doing data prep in the cloud means it is always on, doesn't require any technical installation, and lets teams collaborate on the work for faster results.

Additionally, a good, cloud-native data preparation tool will offer other benefits (like an intuitive and simple-to-use GUI) for easier and more efficient preparation.

Data preparation steps

The specifics of the data preparation process vary by industry, organization, and need, but the workflow remains largely the same.

1. Gather data

The data preparation process begins with finding the right data. This can come from an existing data catalog or data sources can be added ad-hoc.

2. Discover and assess data

After collecting the data, it is important to discover each dataset. This step is about getting to know the data and understanding what has to be done before the data becomes useful in a particular context.

Discovery is a big task, but Talend's data preparation platform offers visualization tools which help users profile and browse their data

3. Cleanse and validate data

Cleaning up the data is traditionally the most time-consuming part of the data preparation process, but it's crucial for removing faulty data and filling in gaps. Important tasks here include:

- Removing extraneous data and outliers
- Filling in missing values
- Conforming data to a standardized pattern
- Masking private or sensitive data entries

Once data has been cleansed, it must be validated by testing for errors in the data preparation process up to this point. Often, an error in the system will become apparent during this validation step and will need to be resolved before moving forward.

4. Transform and enrich data

Data transformation is the process of updating the format or value entries in order to reach a well-defined outcome, or to make the data more easily understood by a wider audience.
Enriching data refers to adding and connecting data with other related information to provide deeper insights.

5. Store data

Once prepared, the data can be stored or channeled into a third party application — such as a business intelligence tool — clearing the way for processing and analysis to take place.

2. Describe different facets of data for data science with examples.

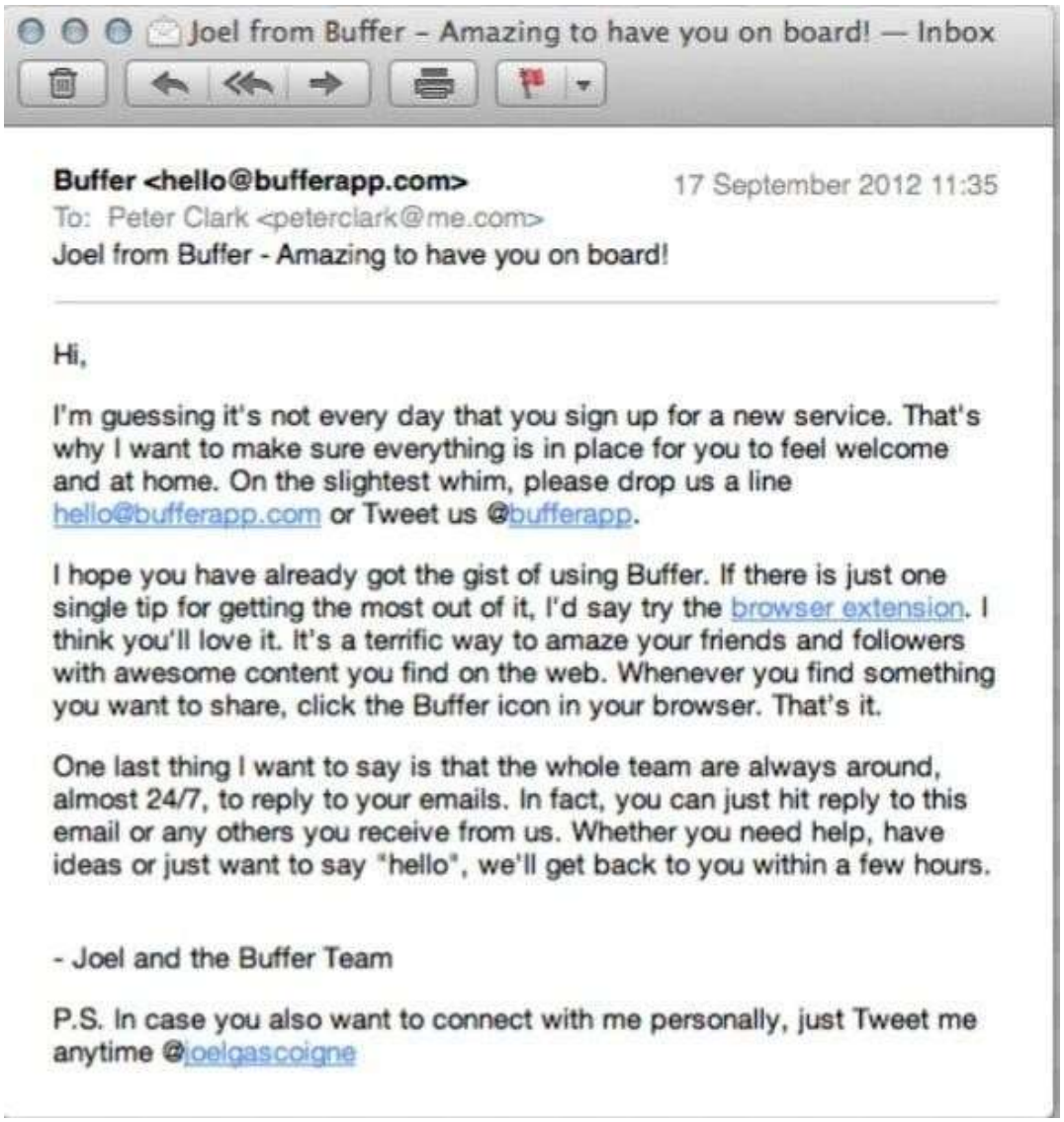
- Structured
- Unstructured
- Natural Language
- Machine-generated
- Graph-based
- Audio, video and images
- Streaming

Structured Data

1	Indicator ID	Dimension List	Timeframe	Numeric Value	Missing Value Flag	Confidence Inte
2	214390830	Total (Age-adjusted)	2008	74.6%		73.8%
3	214390833	Aged 18-44 years	2008	59.4%		58.0%
4	214390831	Aged 18-24 years	2008	37.4%		34.6%
5	214390832	Aged 25-44 years	2008	66.9%		65.5%
6	214390836	Aged 45-64 years	2008	88.6%		87.7%
7	214390834	Aged 45-54 years	2008	86.3%		85.1%
8	214390835	Aged 55-64 years	2008	91.5%		90.4%
9	214390840	Aged 65 years and over	2008	94.6%		93.8%
10	214390837	Aged 65-74 years	2008	93.6%		92.4%
11	214390838	Aged 75-84 years	2008	95.6%		94.4%
12	214390839	Aged 85 years and over	2008	96.0%		94.0%
13	214390841	Male (Age-adjusted)	2008	72.2%		71.1%
14	214390842	Female (Age-adjusted)	2008	76.8%		75.9%
15	214390843	White only (Age-adjusted)	2008	73.8%		72.9%
16	214390844	Black or African American only (Age-adjusted)	2008	77.0%		75.0%
17	214390845	American Indian or Alaska Native only (Age-adjusted)	2008	66.5%		57.1%
18	214390846	Asian only (Age-adjusted)	2008	80.5%		77.7%
19	214390847	Native Hawaiian or Other Pacific Islander only (Age-adjusted)	2008	DSU		
20	214390848	2 or more races (Age-adjusted)	2008	75.6%		69.6%

Fig : An Excel Sheet is an example of Structured Data

Structured data is the data that depends on a data model and resides in a fixed field within a record. It's often easy to store structured data in tables within data bases or Excel files.
SQL, Structured Query Language, is the preferred way to manage and query data that resides in data bases. You may also come across structured data that might give you a hard time storing it in a traditional relational database.
Hierarchical data such as a family tree is one such example.The world isn't made up of structured data, though; it's imposed upon it by humans and machines.



Joel from Buffer - Amazing to have you on board! — Inbox

Buffer <hello@bufferapp.com> 17 September 2012 11:35
To: Peter Clark <peterclark@me.com>
Joel from Buffer - Amazing to have you on board!

Hi,

I'm guessing it's not every day that you sign up for a new service. That's why I want to make sure everything is in place for you to feel welcome and at home. On the slightest whim, please drop us a line hello@bufferapp.com or Tweet us @bufferapp.

I hope you have already got the gist of using Buffer. If there is just one single tip for getting the most out of it, I'd say try the browser extension. I think you'll love it. It's a terrific way to amaze your friends and followers with awesome content you find on the web. Whenever you find something you want to share, click the Buffer icon in your browser. That's it.

One last thing I want to say is that the whole team are always around, almost 24/7, to reply to your emails. In fact, you can just hit reply to this email or any others you receive from us. Whether you need help, have ideas or just want to say "hello", we'll get back to you within a few hours.

- Joel and the Buffer Team

P.S. In case you also want to connect with me personally, just Tweet me anytime @joelgascoigne

Unstructured Data

Unstructured data is data that isn't easy to fit into a data model because the content is context-specific or varying. One example of unstructured data is your regular email. Although email contains structured elements such as the sender, title, and body text, it's a challenge to find the number of people who have written an email complaint about a specific employee because so many ways exist to refer to a person, for example. The thousands of different languages and dialects out there further complicate this.
A human-written email, is also a perfect example of natural language data.

Natural Language

Natural language is a special type of unstructured data ;it's challenging to process because it requires knowledge of specific data science techniques and linguistics.

The natural language processing community has had success in entity recognition, topic recognition, summarization, text completion, and sentiment analysis, but models trained in one domain don't generalize well to other domains. Even state-of-the-art techniques aren't able to decipher the meaning of every piece of text. This shouldn't be a surprise though: humans struggle with natural language as well. It's ambiguous by nature. The concept of meaning itself is questionable here. Have two people listen to the same conversation. Will they get the same meaning? The meaning of the same words can vary when coming from someone upset or joyous.

Machine-generated Data

Machine-generated data is informative that's automatically created by a computer, process, application or other machine without human intervention. Machine-generated data is becoming a major data resource and will continue to do so.

The analysis of Machine data relies on highly scalable tools, due to high volume and speed. Examples are, web server logs, call detail records, network event logs and telemetry.

```
CSIPERF:TXCOMMIT;313236
2014-11-28 11:36:13, Info                  CSI    00000153 Creating NT transaction (seq
69), objectname [6]"(null)"
2014-11-28 11:36:13, Info                  CSI    00000154 Created NT transaction (seq 69)
result 0x00000000, handle @0x4e54
2014-11-28 11:36:13, Info                  CSI    00000155@2014/11/28:10:36:13.471
Beginning NT transaction commit...
2014-11-28 11:36:13, Info                  CSI    00000156@2014/11/28:10:36:13.705 CSI perf
trace:
CSIPERF:TXCOMMIT;273983
2014-11-28 11:36:13, Info                  CSI    00000157 Creating NT transaction (seq
70), objectname [6]"(null)"
2014-11-28 11:36:13, Info                  CSI    00000158 Created NT transaction (seq 70)
result 0x00000000, handle @0x4e5c
2014-11-28 11:36:13, Info                  CSI    00000159@2014/11/28:10:36:13.764
Beginning NT transaction commit...
2014-11-28 11:36:14, Info                  CSI    0000015a@2014/11/28:10:36:14.094 CSI perf
trace:
CSIPERF:TXCOMMIT;386259
2014-11-28 11:36:14, Info                  CSI    0000015b Creating NT transaction (seq
71), objectname [6]"(null)"
2014-11-28 11:36:14, Info                  CSI    0000015c Created NT transaction (seq 71)
result 0x00000000, handle @0x4e5c
2014-11-28 11:36:14, Info                  CSI    0000015d@2014/11/28:10:36:14.106
Beginning NT transaction commit...
2014-11-28 11:36:14, Info                  CSI    0000015e@2014/11/28:10:36:14.428 CSI perf
trace:
CSIPERF:TXCOMMIT;375581
```

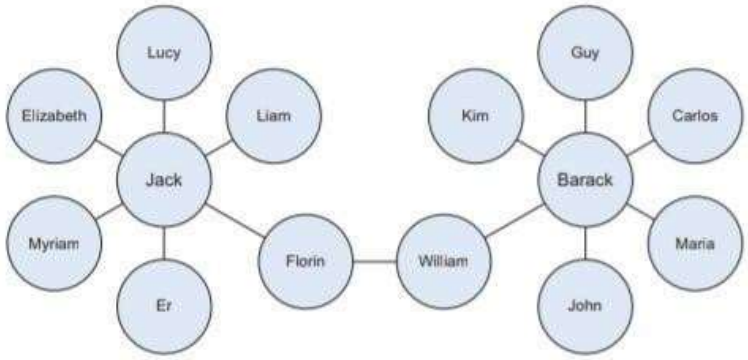
Example for Machine data

This is not the best approach for highly interconnected or "networked" data, where the relationship between entities have a valuable role to play.

Graph-based or Network Data

“Graph data” can be a confusing term because any data can be shown in a graph. “Graph” in this case points to mathematical graph theory. In graph theory, a graph is a mathematical structure to model pair-wise relationships between objects. Graph or network data is, in short, data that focuses on the relationship or adjacency of objects.

The graph structures use nodes, edges, and properties to represent and



store graphical data.

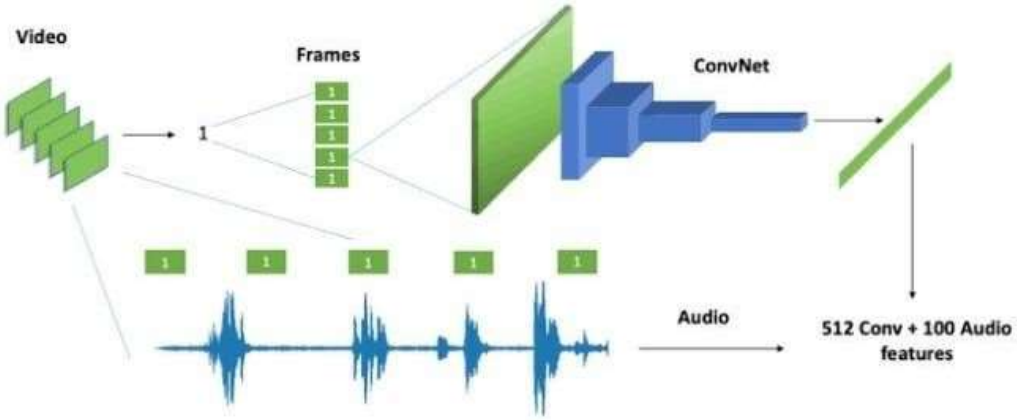
Friends in social network is an example of Graph-based data

Graph-based data is a natural way to represent social networks, and its structure allows you to calculate specific metrics such as the influence of a person and the shortest path between two people.

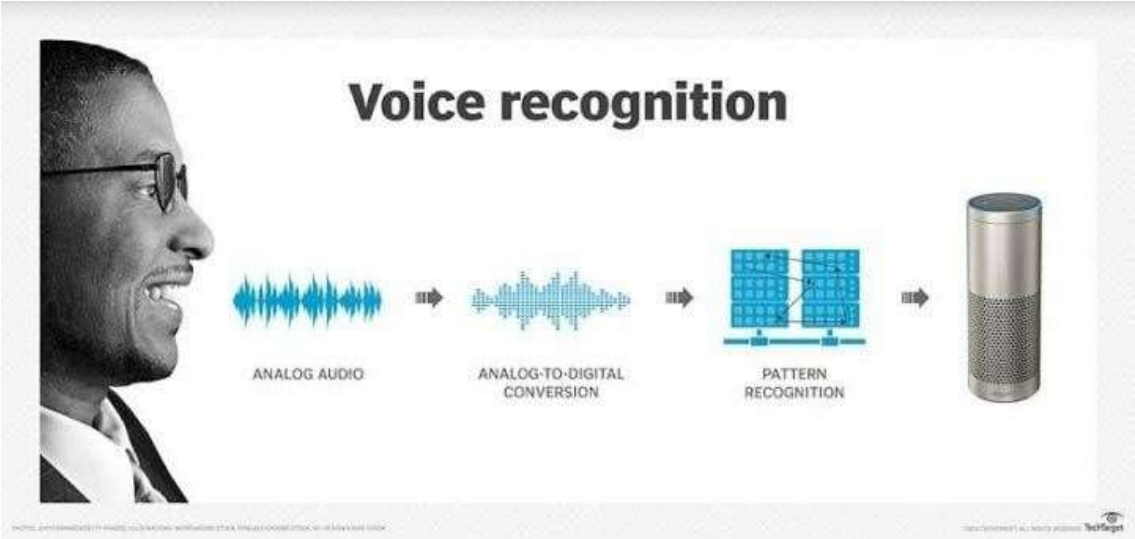
Graph databases are used to store graph-based data and are queried with specialized query languages such as SPARQL.

Graph data poses its challenges, but for a computer interpreting additive and image data, it can be ever more difficult.

Audio, Images and Videos



Audio, image, and video are data types that pose specific challenges to a data scientist. Tasks that are trivial for humans, such as recognizing objects in pictures, turn out to be challenging for computers. Multimedia data in the form of audio, video, images and sensor signals have become an integral part of everyday life. Moreover, they have revolutionized product testing and evidence collection by providing



multiple sources of data for quantitative and systematicassessment.

We have various libraries, development languages and IDEs commonly used in the field, such as :

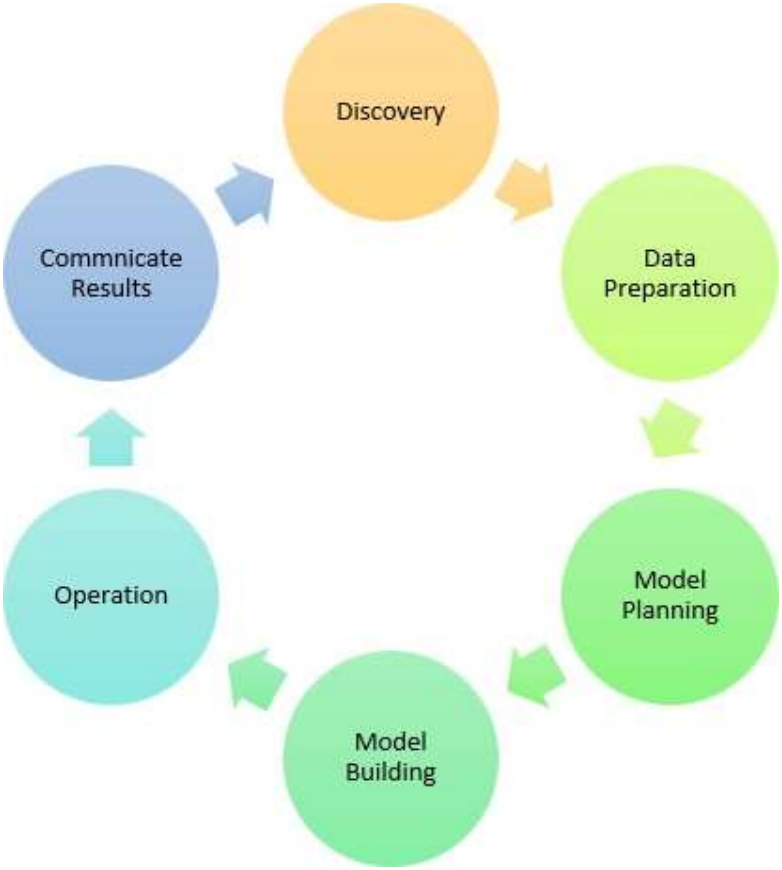
- MATLAB
- openCV
- ImageJ
- Python
- R
- Java
- C
- C++
- C#

Streaming Data

While streaming data can take almost any of the previous forms, it has an extra property. The data flows into the system when an event happens instead of being loaded into a data store in a batch. Although it isn't really a different type of data, we treat it here as much because you need to adapt your process to deal with this type of information. Examples are the "What's trending" on Twitter, live sporting or music events and the stock market.

3. Elaborate the data science process with a neat sketch.

Data Science Process



1. Discovery:

Discovery step involves acquiring data from all the identified internal& external sources, which helps you answer the business question. The data can be:

- Logs from webservers
- Data gathered from social media
- Census datasets
- Data streamed from online sources using APIs

2. Preparation:

Data can have many inconsistencies like missing values, blank columns, an incorrect data format, which needs to be cleaned. You

need to process, explore, and condition data before modelling. The cleaner your data, the better are your predictions.

3. Model Planning:

In this stage, you need to determine the method and technique todraw the relation between input variables. Planning for a model isperformed by using different statistical formulas and visualization tools. SQL analysis services, R, and SAS/access are some of thetools used for this purpose.

4. Model Building:

In this step, the actual model building process starts. Here, Data scientist distributes datasets for training and testing. Techniques like association, classification, and clustering are applied to the training data set. The model, once prepared, is tested against the “testing” dataset.

5. Operationalize:

You deliver the final baselined model with reports, code, and technical documents in this stage. Model is deployed into a real-time production environment after thorough testing.

6. Communicate Results

In this stage, the key findings are communicated to all stakeholders.This helps you decide if the project results are a success or a failurebased on the inputs from the model

4. Describe the basic array operations with examples.

The basic operations in the Arrays are insertion, deletion, searching, display, traverse, and update.

Insertion Operation

In the insertion operation, we are adding one or more elements to the array. Based on the requirement, a new element can be added at the beginning, end, or any given index of array. This is done using input statements of the programming languages.

Deletion Operation

In this array operation, we delete an element from the particular index of an array. This deletion operation takes place as we assign the value in the consequent index to the current index

Search Operation

Searching an element in the array using a key; The key element sequentially compares every value in the array to check if the key is present in the array or not.

Traversal Operation

This operation traverses through all the elements of an array. We use loop statements to carry this out.

Update Operation

Update operation refers to updating an existing element from the array at a given index.

Display Operation

This operation displays all the elements in the entire array using a print statement.

5. **Discuss in detail any real time application of data science with examples.**

1. **In Search Engines**

The most useful application of Data Science is Search Engines. As we know when we want to search for something on the internet, we mostly used Search engines like Google, Yahoo, Safari, Firefox, etc. So Data Science is used to get Searches faster.

For Example, When we search something suppose “Data Structure and algorithm courses ” then at that time on the Internet Explorer we get the first link of GeeksforGeeks Courses.
This happens because the GeeksforGeeks website is visited most in order to get information regarding Data Structure courses and Computer related subjects. So this analysis is Done using Data Science, and we get the Topmost visited Web Links.

2. **In Transport**

Data Science also entered into the Transport field like Driverless Cars. With the help of Driverless Cars, it is easy to reduce the number of Accidents.

For Example, In Driverless Cars the training data is fed into the algorithm and with the help of Data Science techniques, the Data is analyzed like what is the speed limit in Highway, Busy Streets, Narrow Roads, etc. And how to handle different situations while driving etc.

3. **In Finance**

Data Science plays a key role in Financial Industries. Financial Industries always have an issue of fraud and risk of losses. Thus, Financial Industries needs to automate risk of loss analysis in order to carry out strategic decisions for the company. Also, Financial Industries uses Data Science Analytics tools in order to predict the future. It allows the companies to predict customer lifetime value and their stock market moves.

For Example, In Stock Market, Data Science is the main part. In the Stock Market, Data Science is used to examine past behavior with past data and their goal is to examine the future outcome. Data is analyzed in such a way that it makes it possible to predict futurestock prices over a set timetable.

4. In E-Commerce

E-Commerce Websites like Amazon, Flipkart, etc. uses data Science to make a better user experience with personalized recommendations.

For Example, When we search for something on the E-commerce websites we get suggestions similar to choices according to our past data and also we get recommendations according to most buy the product, most rated, most searched, etc. This is all done with the help of Data Science.

5. In Health Care

In the Healthcare Industry data science act as a boon. Data Science is used for: Detecting Tumor.
Drug discoveries.
Medical Image
Analysis. Virtual
Medical Bots.

Genetics and Genomics.
Predictive Modeling for Diagnosis etc.

6. Image Recognition

Currently, Data Science is also used in Image Recognition. For Example, When we upload our image with our friend on Facebook, Facebook gives suggestions Tagging who is in the picture. This is done with the help of machine learning and Data Science. When an Image is Recognized, the data analysis is done on one's Facebook friends and after analysis, if the faces which are present in the picture matched with someone else profile then Facebook suggests us auto-tagging.

7. Targeting Recommendation

Targeting Recommendation is the most important application of Data Science. Whatever theuser searches on the Internet, he/she will see numerous posts everywhere. This can be explained properly with an example: Suppose I want a mobile phone, so I just Google search it and after that, I changed my mind to buy offline. Data Science helps those companies whoare paying for Advertisements for their mobile. So everywhere on the internet in the social media, in the websites, in the apps everywhere I will see the recommendation of that mobile phone which I searched for. So this will force me to buy online.

8. Airline Routing Planning

With the help of Data Science, Airline Sector is also growing like with the help of it, it becomes easy to predict flight delays. It also helps to decide whether to directly land into the destination or take a halt in between like a flight can have a direct route from Delhi to the U.S.A or it can halt in between after that reach at the destination.

9. Data Science in Gaming

In most of the games where a user will play with an opponent i.e. a Computer Opponent, data science concepts are used with machine learning where with the help of past data the Computer will improve its performance. There are many games like Chess, EA Sports, etc. will use Data Science concepts.

10. Medicine and Drug Development

The process of creating medicine is very difficult and time-consuming and has to be done with full disciplined because it is a matter of Someone's life. Without Data Science, it takes lots of time, resources,

and finance or developing new Medicine or drug but with the help of Data Science, it becomes easy because the prediction of success rate can be easily determined based on biological data or factors. The algorithms based on data science will forecast how this will react to the human body without lab experiments.

11. In Delivery Logistics

Various Logistics companies like DHL, FedEx, etc. make use of Data Science. Data Science helps these companies to find the best route for the Shipment of their Products, the best time suited for delivery, the best mode of transport to reach the destination, etc.

12. Autocomplete

AutoComplete feature is an important part of Data Science where the user will get the facility to just type a few letters or words, and he will get the feature of auto-completing the line. In

Google Mail, when we are writing formal mail to someone so at that time data science concept of the Autocomplete feature is used where he/she is an efficient choice to auto-complete the whole line. Also in Search Engines in social media, in various apps, AutoComplete feature is widely used.

6. Describe indexing and slicing numpy arrays

NumPy is a powerful library that provides support for arrays, matrices, and a wide range of mathematical operations. NumPy arrays are multi-dimensional data structures that can beindexed and sliced to extract or manipulate subsets of data. Indexing refers to accessing individual elements of an array, while slicing involves extracting a portion of the array by specifying a range of indices.

Indexing:

Indexing in NumPy arrays is similar to indexing in standard Python lists, but it extends naturally to multiple dimensions. NumPy arrays can have any number of dimensions, and each dimension is accessed using an index corresponding to that dimension.

For a 1D array:

python
```
import numpy as np
arr_1d = np.array([0, 1, 2, 3, 4])
element = arr_1d[2]  # Access the element at index
```
2 (value: 2)For a 2D array:

python
```
arr_2d = np.array([[0, 1, 2], [3, 4, 5]])
element = arr_2d[1, 2]  # Access the element at row 1, column 2 (value: 5)
```

Slicing:
Slicing allows you to extract a portion of an array by specifying a range of indices for each dimension. Slicing in NumPy returns a new view of the array, not a copy, which means modifying the slice will also affect the original array.

For a 1D array:

python
```
arr_1d = np.array([0, 1, 2, 3, 4])
subset = arr_1d[1:4]  # Slice elements from index 1 to 3
```
(values: [1, 2, 3])For a 2D array:

python

```
arr_2d = np.array([[0, 1, 2], [3, 4, 5], [6, 7, 8]])
subset = arr_2d[0:2, 1:3]  # Slice rows 0 to 1 and columns 1 to 2
# Resulting subset: [[1, 2], [4,
5]] Slicing also supports the
use of steps:
```

python

```
arr_1d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
subset = arr_1d[1:9:2]  # Slice with step 2 (values: [1, 3, 5, 7])
```

Remember that the end index in slicing is exclusive, meaning the element at the end index is not included in the slice.

Both indexing and slicing are fundamental operations for working with NumPy arrays, allowing you to access and manipulate data efficiently and effectively.

7. Elaborate the following and give examples
a. Numpy Random: Data Distribution, Normal
b. Exponential distribution
c. Binomial distribution
d. Poisson, Uniform and
e. Chi Square distributions

In the context of NumPy, the "Random" module refers to the set of functions thatallow you to generate random numbers and data with various probability distributions. These functions are useful for simulating random processes, conducting statistical experiments, and generating synthetic data for various purposes.

Here are some common probability distributions you can work with using NumPy'srandom functions:

<u>Uniform Distribution:</u>
Generates random numbers uniformly distributed between a specified range. Example: Rolling a fair six-sided die, where each outcome has an equal probabilityof 1/6.

python
```
import numpy as np
uniform_data = np.random.uniform(low=0, high=1, size=10)
```

<u>Normal (Gaussian) Distribution:</u>
Generates random numbers following a bell-shaped curve characterized by mean (μ)and standard deviation (σ).
Example: Heights of a population.

python
```
normal_data = np.random.normal(loc=0, scale=1, size=1000)
```

Binomial Distribution:
Generates random numbers representing the number of successes in a fixednumber of independent Bernoulli trials.
Example: Tossing a coin multiple times and counting the number of heads.

python
```
binomial_data = np.random.binomial(n=10, p=0.5, size=100)
```

Poisson Distribution:
Generates random numbers representing the number of events occurring in a fixedinterval of time or space.
Example: Number of emails received per hour.

python
```
poisson_data = np.random.poisson(lam=3, size=50)
```

Exponential Distribution:
Generates random numbers representing the time between events in a Poissonprocess.
Example: Time between arrivals of customers at a store.

python
```
exponential_data = np.random.exponential(scale=2, size=200)
```

Chi-square Distribution
The Chi-Square distribution is commonly used in statistics for hypothesis testing andconfidence interval calculations. In NumPy, you can easily generate random numbers following a Chi-Square distribution using the numpy.random module. Here'show you can do it:

python

```
import numpy as np

# Degrees of freedom (shape
parameter)df = 5

# Generate random numbers from a Chi-Square
distributionchi_square_data =
np.random.chisquare(df, size=100)
```

```
print(chi_square_data)
```

In this example, df represents the degrees of freedom, which determines the shapeof the Chi-Square distribution. The size parameter specifies how many random numbers you want to generate.

Remember to replace '5' with your desired degrees of freedom and adjust the size asneeded for your application. The generated chi_square_data will be an array containing random numbers that follow the Chi-Square distribution with the specified degrees of freedom.

PART C

1. Develop a python program to create a 3D array of size 3x4x4 with all elements initialised to 25. Display its dimension, shape, size and data. Code:

```
import numpy as np

# Create a 3D array of size 3x4x4 with all elements
initialised to 25array_3d = np.full((3, 4, 4), 25)

# Display dimension
dimension =
array_3d.ndim

# Display shape
shape = array_3d.shape

# Display size
size = array_3d.size

# Display array
datadata =
array_3d

print("3D Array Dimension:",
dimension)print("3D Array
Shape:", shape) print("3D Array
Size:", size)
print("3D Array Data:\n", data)
```

OUTPUT:

```
3D Array
Dimension: 3 3D
Array Shape: (3, 4,
4)3D Array Size:
48
3D Array
 Data:[[[25
 25 25 25]
  [25 25 25 25]
  [25 25 25 25]
  [25 25 25 25]]
```

[[25 25 25 25]

```
 [25 25 25 25]
 [25 25 25 25]
 [25 25 25 25]]

[[25 25 25 25]
 [25 25 25 25]
 [25 25 25 25]
 [25 25 25 25]]]
```

2. Develop a python program to generate a 1-D array containing 100 random values, where each value has to be 3, 5, 7 or 9 and their probabilities are 0.1,0.3, 0.4, 0.2 respectively.

Code:

```
import numpy as np

# Values to choose
fromvalues = [3, 5,
7, 9]

# Probabilities corresponding to the
valuesprobabilities = [0.1, 0.3, 0.4,
0.2]

# Generate a 1-D array of size 100 with values based on
specified probabilitiesrandom_array = np.random.choice(values,
size=100, p=probabilities)

print("Generated Random
Array:")print(random_array)
```

OUTPUT:

```
Generated Random Array:
[5 7 7 7 5 9 5 7 7 9 7 5 7 3 5 7 9 5 5 7 9 7 9 7 9 7 7 7 9 7 7 5 7 7 7 7 7 5 7
7 9 7 7 9 7 5 7 7 7
7 9 7 7 7 7 9 7 7 7 5 9 9 9 7 7 9 5 7 9 7 5 9 9 9 7 7 9 9 7 7 7 7 7 9 7 5 7
7 9 5 9 9 9 5 7 7 7
7 7 7]
```

3. Develop a python program to create an array of all odd numbers between 0and 500. Update every 4th element with its square value.

Code:

```
import numpy as np

# Create an array of odd numbers between 0 and
500odd_numbers = np.arange(1, 500, 2)

# Update every 4th element with its square value
odd_numbers[3::4] = odd_numbers[3::4] ** 2

print("Array of Odd Numbers with Squared 4th
Elements:")print(odd_numbers)
```

Output:

```
Array of Odd Numbers with Squared 4th Elements:
[   1   3   5   16  9  35   49   64  121  169  225  289  361  441  529
  625  729  841  961 1089 1225 1369 1521 1681 1849 2025 2209 2401
  2601 2809
 3025 3249 3481 3721 3969 4225 4489 4761 5041 5329 5625 5929 6241
 6561 6889
 7225 7569 7921 8281 8649 9025 9409 9801]
```

4. Develop a python program to draw out a sample for exponential distributionwith 5.0 scale with 4x3 size.
Code:

```
import numpy as np

# Parameters for the exponential
distributionscale = 5.0
size = (4, 3)

# Generate a sample from the exponential
distributionsample =
np.random.exponential(scale, size=size)

print("Sample from Exponential
Distribution:")print(sample)
```

Output:

```
Sample from Exponential Distribution:
[[ 4.57924596 13.83278513  2.12240559]
 [ 4.43061501 14.87483697  8.57593594]
 [ 6.0600027  0.47641314  5.58898765]
 [ 1.8869717  5.25359271  5.99457047]]
```

5.. Develop a python program to multiply 2 NumPy array matricesCode:

```
import numpy as
np # NumPy
matrices
A =
np.arange(15,24).reshape(3,
3) B =
np.arange(20,29).reshape(3,
3)print("A: ",A)
print("B: ",B)
# Multiply A and B
result = A.dot(B)
print("Result: ",
result)
```

Output:

```
A: [[15 16 17]
[18 19 20]
[21 22 23]]
B: [[20 21 22]
[23 24 25]
[26 27 28]]
Result: [[1110 1158 1206]
[1317 1374 1431]
[1524 1590 1656]]
```

6. Demonstrate with example vstack() is different from hstack() in NumPy In NumPy, vstack() and hstack() are functions used to stack arrays vertically andhorizontally, respectively. They are commonly used for combining arrays along different dimensions. Let's demonstrate their differences with examples:

Example 1: Using vstack() for Vertical StackingCode:

```
import numpy as np

# Create two arrays
array1 = np.array([[1, 2, 3]])
array2 = np.array([[4, 5, 6]])

# Vertically stack the arrays
vertical_stack = np.vstack((array1, array2))

print("Array
1:")
print(array1)
print("Array
2:")
print(array2)
print("Vertically Stacked
Array:")print(vertical_stack)
```

Output:

```
Array 1:
[[1 2 3]]
Array 2:
[[4 5 6]]
Vertically Stacked Array:
[[1 2 3]
 [4 5 6]]
```

In this example, the vstack() function stacked array1 and array2 vertically to create anew array with two rows.

Example 2: Using hstack() for Horizontal StackingCode:

```
import numpy as np
```

```
# Create two arrays
array1 = np.array([[1, 2], [3, 4]])
array2 = np.array([[5, 6],

[7, 8]])# Horizontally stack

the arrays
```

```
horizontal_stack = np.hstack((array1, array2))

print("Array
1:")
print(array1)
print("Array
2:")
print(array2)
print("Horizontally Stacked
Array:")print(horizontal_stack)
```

Output:

```
Array 1:
[[1 2]
 [3 4]]
Array 2:
[[5 6]
 [7 8]]
Horizontally Stacked Array:
[[1 2 5 6]
 [3 4 7 8]]
```

In this example, the hstack() function stacked array1 and array2 horizontally tocreate a new array with additional columns.

Summary:

vstack() is used for stacking arrays vertically, increasing the number of rows. hstack() is used for stacking arrays horizontally, increasing the number of columns.These functions are useful when you want to combine arrays to form larger arrays for various purposes, such as data preprocessing, feature engineering, and more complex data manipulations.

7. **Both methods are used for combining the NumPy arrays. The main difference is that the hstack method combines arrays horizontally whereas thevstack method combines arrays vertically.**

For example, consider the below code.

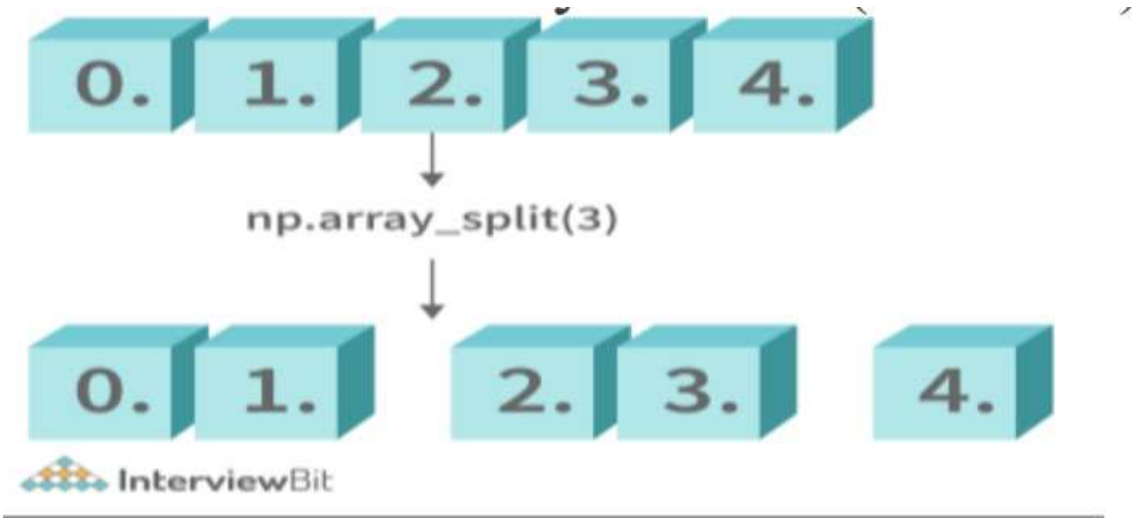
```
import numpy as np
a = np.array([1,2,3])
b = np.array([4,5,6])

# vstack arrays
c = np.vstack((a,b))
print("After vstack:
\n",c)# hstack arrays
d = np.hstack((a,b))
print("After hstack:
\n",d)
```

The output of this code would be:

```
After vstack:
[[1 2 3] [4 5 6]]
After
hstack:[1 2
3 4 5 6]
```

8. What happens when we use the arrays_split() method for splitting theNumPy array?

The array_split() method is similar to the split() method as it helps in splitting a given array to multiple subarrays. The main difference is that the array_split() allows sections to be an integer which does not result in equal array division. For an array of length L, if we want it to split to N subarrays, then L % N subarrays of size (L//N + 1) and remaining subarrays are of size L//N.



In the above figure, we see there are 5 elements in the array, we want to split the array to 3 subarrays. So L % N = 5%3 = 2 subarrays of size (L//N +1) = (5//3 +1) = 2 are returned and remaining 1 subarray of size L//N = 1 is returned.

Syntax:
np.array_split(array, sections,
axis=0)where,
array - Given Input array.
sections - List of indices or Number of subarrays to
be returned.axis - Axis along which values have to
be appended.
The code for the example illustrated
above is:import numpy as np
arr = np.arange(5.0)

split_arrs = np.array_split(arr,
3)split_arrs Output: [array([0.,
1.]),
array([2., 3.]), array([4.])]

UNIT -2

PART B

1. Explain the CRUD operations with SQLite

CRUD stands for Create, Read, Update, and Delete, which are the four fundamental operations for managing data in a database. You can perform these operations in SQLite, a popular embedded relational database management system (RDBMS), using SQL statements or various programming languages such as Python, Java, or C++. Here's an explanation of each CRUD operation in the context of SQLite:

1. Create (C):
   - To create a new record (row) in an SQLite table, you use the `INSERT` statement.

   Syntax;
   Insert_into_ table_name (column1, column2, column3, ...)
   values(value1, value2, value3, ...);

2. Read (R)
   - To retrieve data from an SQLite table, you use the `SELECT` statement.
   Syntax;
   SELECT column1, column2, ...
   FROM table_name
   WHERE condition;

   3. Update (U):
   - To modify existing records in an SQLite table, you use the `UPDATE` statement.
   Syntax;
   UPDATE table_name
   SET column1 = value1, column2 = value2, ...

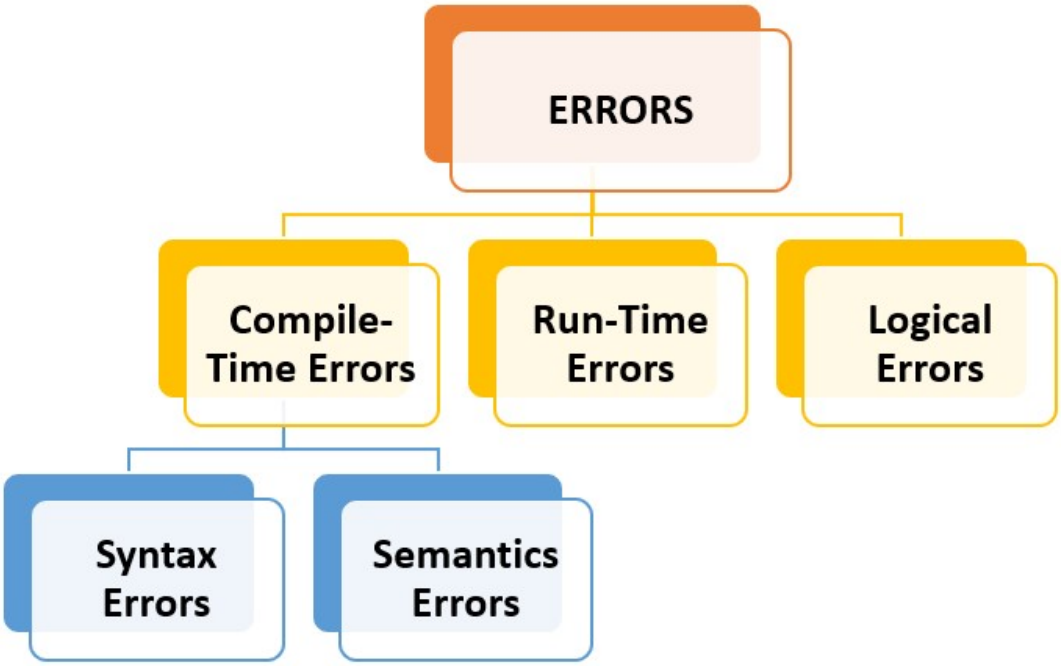
WHERE condition;

4. **Delete (D)**:
   - To remove records from an SQLite table, you use the `DELETE` statement.

Syntax;
DELETE FROM table_name
WHERE condition;

These are the basic CRUD operations in SQLite. They allow you to create, retrieve, update, and delete data in your SQLite database, making it a powerful tool for managing structured data in applications ranging from simple data storage to more complex data-driven applications.

2. Classify different types of error and give examples.



Compile-time Errors:

Description: Compile-time errors, also known as syntax errors, occur during the compilation phase of a program, before it is executed. These errors occur when the code does not adhere to the correct syntax or structure of the programming language.

Examples:

Missing a semicolon at the end of a statement in C++.

Using an undeclared variable in Python.
Mismatched parentheses in an arithmetic expression.
Significance: Compile-time errors prevent the program from being successfully compiled, and the program cannot be executed until these errors are fixed. They are typically detected by the compiler, and you'll receive error messages pointing to the specific issues in your code.

Run-time Errors (Exceptions):

Description: Run-time errors occur during the execution of a program. They occur when the program encounters unexpected conditions, such as division by zero, attempting to access an invalid memory location, or trying to open a non-existent file.

Examples:

Division by zero in arithmetic operations.
Attempting to access an element outside the bounds of an array.
Opening a file that doesn't exist.
Significance: Run-time errors cause the program to terminate abruptly. They need to be handled properly using error handling mechanisms (e.g., try-except blocks in Python) to prevent the program from crashing and to provide graceful error messages.

Logical Errors (Semantic Errors):

Description: Logical errors, also known as semantic errors, occur when the program's logic is flawed. The code is syntactically correct, but it does not produce the expected results due to incorrect algorithms or incorrect assumptions about how the program should behave.

Examples:

Incorrect formula for calculating the area of a circle.
Flawed logic in an algorithm that leads to incorrect results.
Misunderstanding of the program's requirements, leading to incorrect behavior.
Significance: Logical errors are the most subtle type of error. They don't produce error messages or crashes but can lead to incorrect program output. Finding and fixing logical errors often requires careful code review, testing, and debugging.

Syntax Errors:

Description: Syntax errors are a subset of compile-time errors. They occur specifically due to violations of the language's syntax rules. Syntax errors prevent the code from being compiled or interpreted.

Examples:

Missing parentheses in a function call.
Incorrect indentation in Python.
Using reserved keywords as variable names.
Significance: Syntax errors are the easiest to detect and fix because they are reported by the compiler or interpreter. They are essential for ensuring that your code follows the correct language rules.

3.Demonstrate the usage of Python Standard Debugger (PDB) with an example.
we'll create a simple Python script that calculates the factorial of a number and debug it using PDB.

Example:
```
import pdb

def factorial(n):
    if n == 0:
        return 1
    else:
        result = n * factorial(n - 1)
        return result

# Set a breakpoint using pdb.set_trace()
pdb.set_trace()

number = 5
print(f"Factorial of {number} is {factorial(number)}")
```

Here's how to use PDB effectively in this example:

1. **Import the `pdb` Module**:
   Start by importing the `pdb` module at the beginning of your script.

2. **Insert a Breakpoint**:
   To set a breakpoint in your code, use the `pdb.set_trace()` function. Place this function call where you want your code execution to pause for debugging. In this example, we've set a breakpoint just before the `number` variable is calculated.

3. **Run Your Program with PDB**:
   Run your Python script as you normally would:

   When the `pdb.set_trace()` line is encountered, your program will pause, and you will enter the PDB debugger shell. You'll see a `(Pdb)` prompt.

4. **PDB Commands**:
   You can use various PDB commands to inspect variables, evaluate expressions, and control program flow.

   - Type `n` or `next` to execute the current line and move to the next line of code.
   - Type `p` or `print` followed by a variable name to print its value.
   - Type `c` or `continue` to continue execution until the next breakpoint is encountered.

5. **Exiting PDB**:
   To exit PDB and finish program execution, you can type `q` or `quit`.

In this example, PDB allows you to interactively debug the `factorial` function, inspect variables (`n` and `result`), and step through the code. It's a powerful tool for diagnosing issues and understanding the flow of your Python programs.

1. Printing Variables or expressions
2. Moving in code by steps
3. Using Breakpoints
4. Execute code until the specified line
5. List the code
6. Displaying Expressions
7. Frames up-down

4. Exhibit the significance of unittest with example.

Unit testing is a method for testing software that looks at the smallest testable pieces of code, called units, which are tested for correct operation. By doing unit testing, we can verify that each part of the code, including helper functions that may not be exposed to the user, works correctly and as intended.

The idea is that we are independently checking each small piece of our program to ensure that it works. This contrasts with regression and integration testing, which tests that the different parts of the program work well together and as intended

The Python standard library includes the unittest module to help you write and run tests for your Python code.

Tests written using the unittest module can help you find bugs in your programs, and prevent regressions from occurring as you change your code over time. Teams adhering to test-driven development may find unittest useful to ensure all authored code has a corresponding set of tests.

5. Develop a program in python to demonstrate the basic functions of regular expressions.

```
import re

# Sample text
text = "Hello, my email is example@example.com and my phone number
is 123-456-7890."

# 1. Searching for an email address:
match = re.search(r'\S+@\S+', text)
if match:
    print(f"Pattern found: '{match.group()}'")
else:
    print("Pattern not found")

# 2. Matching a name at the beginning of the text:
match = re.match(r'Hello', text)
if match:
    print(f"Pattern matched: '{match.group()}'")
else:
```

```
    print("Pattern not matched at the beginning")

# 3. Finding all phone numbers:
matches = re.findall(r'\d{3}-\d{3}-\d{4}', text)
if matches:
    print(f"All patterns found: {matches}")
else:
    print("No patterns found")

# 4. Substituting email addresses with 'REDACTED':
new_text = re.sub(r'\S+@\S+', 'REDACTED', text)
print(f"Original Text: '{text}'")
print(f"New Text: '{new_text}'")
```

6. Demonstrate the difference between class and instance variables with examples.

Instance Variable	Class Variable

It is a variable whose value is instance-specific and now shared among instances.	It is a variable that defines a specific attribute or property for a class.
These variables cannot be shared between classes. Instead, they only belong to one specific class.	These variables can be shared between class and its subclasses.
It usually reserves memory for data that the class needs.	It usually maintains a single shared value for all instances of class even if no instance object of the class exists.
It is generally created when an instance of the class is created.	It is generally created when the program begins to execute.
It normally retains values as long as the object exists.	It normally retains values until the program terminates.
It has many copies so every object has its own personal copy of the instance variable.	It has only one copy of the class variable so it is shared among different objects of the class.

It can be accessed directly by calling variable names inside the class.	It can be accessed by calling with the class name.
These variables are declared without using the static keyword.	These variables are declared using the keyword static.

Changes that are made to these variables through one object will not reflect in another object.
 	
Example:
 	
class Taxes
 	
{
 	
int count;
 	
/*...*/
 	
}
	Changes that are made to these variables through one object will reflect in another object.
	
	Example:
	
	class Taxes
	
	{
	
	static int count;
	
	/*...*/
	
	}

7. Describe the features of JSON and explain reading, writing and parsing JSON files.

Features of JSON

- Simplicity
- Openness
- Self-Describing
- Internationalization
- Extensibility
- Interoperability

JSON (JavaScript Object Notation) is a widely-used data interchange format that has gained popularity for its simplicity and versatility. It is a text-based format that consists of key-value pairs, arrays, and nested data structures. Here are the key features of JSON:

Simplicity: JSON has a straightforward and easy-to-understand syntax. It uses a minimal set of punctuation marks like braces {}, square brackets [], colons :, and commas , to structure data.

Human-Readable: JSON is human-readable and easy to write by hand. It is presented in a plain text format that is both compact and clear. This makes it accessible for developers and easy to troubleshoot.

Lightweight: JSON is lightweight and efficient, making it an ideal choice for data transmission over the internet. Its simplicity results in minimal overhead, making it suitable for use in web applications and APIs.

Self-Describing: JSON data is self-describing, meaning that it includes information about its structure within the data itself. Objects and arrays can be nested, allowing for the representation of complex data hierarchies.

Data Types: JSON supports several basic data types, including strings, numbers, booleans, null values, arrays, and objects. This versatility allows you to represent a wide range of data structures and values.

Language Independence: JSON is a language-independent format, which means it can be used to exchange data between different programming languages. Nearly all modern programming languages have libraries or built-in support for parsing and generating JSON.

Widely Supported: JSON is supported by a wide range of software and tools, making it a popular choice for data serialization and configuration files in various applications.

Schemaless: JSON is schemaless, meaning it does not require a predefined schema to specify the structure of data. This flexibility can be advantageous when dealing with evolving or dynamic data.

Extensibility: JSON allows you to include custom data types and structures through the use of objects and arrays. This flexibility enables you to adapt JSON to specific application requirements.

Security: JSON is generally considered safe for data exchange because it does not support executable code. However, like any data format, it should be parsed and processed with security precautions to prevent potential vulnerabilities.

Reading JSON:
You can read JSON data from a file using Python's built-in json module. Here's how you can do it:

```
import json

# Read JSON data from a file
with open('data.json', 'r') as file:
    data = json.load(file)

# 'data' now contains the JSON data as a Python dictionary or list
```

Writing JSON:
To write data to a JSON file, you can use the json.dump() function:

```
import json

data = {
    'name': 'John',
    'age': 30,
```

```
    'city': 'New York'
}

# Write data to a JSON file
with open('output.json', 'w') as file:
    json.dump(data, file)
```

This will create a file named output.json containing the JSON representation of the data dictionary.

Parsing JSON:
Once you've loaded JSON data into a Python variable, you can parse and manipulate it as needed. JSON data will be converted into Python dictionaries, lists, strings, numbers, and booleans depending on the structure of the JSON.

8. Establish the role of JSON schema in JSON file validation.

JSON Schema is a powerful tool for validating JSON data. It serves as a specification for defining the structure, data types, and constraints of JSON documents. The role of JSON Schema in JSON file validation is crucial, as it helps ensure the integrity, correctness, and compliance of JSON data with predefined rules. Here are the key roles of JSON Schema in JSON file validation:

Structure Validation: JSON Schema defines the expected structure of a JSON document, including the hierarchy of objects and arrays, the names of keys, and their corresponding data types. It helps verify that a JSON file adheres to the specified structure.

Data Type Validation: JSON Schema allows you to specify the expected data types for values within the JSON document. This ensures that, for example, a field that should contain a number doesn't accidentally contain a string or a boolean.

Value Validation: Beyond data types, JSON Schema enables you to set constraints on the allowable values. For instance, you can define minimum and maximum values for numbers, minimum and maximum lengths for strings, and enumerated lists of acceptable values.

Conditional Validation: JSON Schema supports conditional validation, allowing you to define rules based on the values of other fields. This is useful for enforcing complex validation logic that depends on the context within the JSON document.

Reusable Definitions: JSON Schema supports the definition of reusable schemas that can be referenced within other schemas. This promotes code reuse and maintainability by allowing you to define common validation rules once and reuse them in multiple places.

Documentation: JSON Schema can serve as documentation for the expected structure and validation rules of JSON data. It makes it clear to developers, users, and automated systems what data should look like and how it should be validated.

Integration with Tools: Various tools and libraries support JSON Schema validation, making it easy to integrate validation into your workflow. These tools can validate JSON data against a specified schema and report validation errors.

Validation Errors: When JSON data fails validation against a JSON Schema, the schema provides detailed error messages, indicating exactly where and why the data does not conform to the defined rules. This helps in diagnosing and fixing issues.

Compatibility: JSON Schema is widely supported across different programming languages, making it a versatile choice for JSON validation in diverse environments.

9. Elucidate the limitations of Floating point Arithmetic.

Floating-point arithmetic, which is commonly used to represent and perform calculations with real numbers in computers, has several limitations that can lead to unexpected behavior and inaccuracies in numerical computations. Here are some of the key limitations of floating-point arithmetic:

Limited Precision: Floating-point numbers have finite precision, meaning they can only represent a fixed number of significant digits. This limited precision can result in rounding errors when representing numbers with many significant digits.

Representation Errors: Not all real numbers can be accurately represented as floating-point numbers. Some numbers, such as irrational numbers like π (pi) or numbers with infinite decimal expansions, must be approximated, leading to representation errors.

Loss of Significance: Subtraction of two nearly equal floating-point numbers can result in a significant loss of precision. This is known as "catastrophic cancellation." It occurs because the significant digits of the two numbers nearly cancel each other out.

Non-Associativity: Floating-point arithmetic is not associative, meaning that the order in which operations are performed can affect the result. This can lead to unexpected results when operations are not performed in the expected order.

Rounding Errors: Rounding errors occur when a floating-point number is converted from its internal representation to a decimal representation for display. These errors can accumulate and affect the accuracy of calculations.

Limited Range: Floating-point numbers have a limited range, both for representing very small and very large numbers. Numbers that fall outside this range may be represented as infinity or zero, leading to loss of information.

Inexact Representation of Fractions: Some fractions that are simple in decimal form cannot be precisely represented in binary floating-point format. For example, 1/3 cannot be exactly represented, resulting in rounding errors.

Comparison Issues: Comparing floating-point numbers for equality can be problematic due to the limited precision and representation errors. It's often recommended to use tolerance thresholds when comparing floating-point numbers.

Denormalized Numbers: Very small floating-point numbers close to zero are represented using denormalized numbers, which have reduced precision. Operations involving denormalized numbers can be slower and less accurate.

Platform Dependence: The behavior of floating-point arithmetic can vary between different hardware platforms and programming languages, leading to non-portable code.

10. Explain the importance of Virtual Environment in python.

A virtual environment in Python is a self-contained, isolated environment that allows you to install and manage Python packages independently of the system-wide Python installation. Virtual environments are important for several reasons:

1. **Dependency Isolation:** Virtual environments provide a way to isolate Python projects and their dependencies. Each virtual environment has its own Python interpreter and a separate set of installed packages. This isolation ensures that packages required by one project do not interfere with those of another, preventing version conflicts and compatibility issues.

2. **Version Control:** Virtual environments allow you to specify and control the exact versions of Python and packages used in a project. This is crucial for maintaining consistent development and production environments, ensuring that your code behaves predictably across different setups.

3. **Package Management:** Virtual environments enable you to use tools like `pip` to install, upgrade, and manage Python packages within the isolated environment. This makes it easy to keep track of project-specific dependencies and ensure that your project always uses the correct versions of libraries.

4. **Sandboxing:** Virtual environments provide a safe sandbox for experimenting with Python packages and code without affecting the system-wide Python installation. This is particularly useful for testing and development, where you may want to try out new libraries or configurations without risking system stability.

5. **Cross-Platform Compatibility:** Virtual environments are portable and can be created on different platforms (Windows, macOS, Linux) with the same project requirements. This ensures that your Python code can run consistently across different operating systems.

6. **Cleaner Project Structure:** By encapsulating project-specific dependencies within a virtual environment, your project's structure becomes cleaner and more organized. It's clear which packages are required for the project, making it easier to share code with others and deploy it to different environments.

7. **Security:** Isolating projects in virtual environments can enhance security. It prevents accidental or malicious modification of system-wide Python packages and reduces the risk of security vulnerabilities affecting other projects.

8. **Performance:** Virtual environments can help improve performance by reducing the overhead of managing a large collection of globally installed packages. You can create lightweight environments with only the packages needed for a specific project.

9. **Conda Environments:** Apart from Python's built-in virtual environment module (`venv`), tools like Conda offer powerful environment management capabilities. Conda allows you to create and manage virtual environments with packages not limited to Python, making it suitable for data science and scientific computing.

In summary, virtual environments are essential tools for Python development. They provide a way to manage project dependencies,

maintain consistency across environments, and ensure that your Python code is portable and secure. Whether you're working on a small script or a large-scale application, using virtual environments is considered a best practice in Python development.

PART C

1. Create a class for employee (eno, ename, basic) and display payroll for 2
employees with da=50% and hra=10%. Define constructor and destructor for
the class.

```
class Employee:
    total_employees = 0

    def __init__(self, eno, ename, basic):
        self.eno = eno
        self.ename = ename
        self.basic = basic
        self.da = 0.50 * basic
        self.hra = 0.10 * basic
        Employee.total_employees += 1

    def display_payroll(self):
        total_salary = self.basic + self.da + self.hra
        print(f"Employee Number: {self.eno}")
        print(f"Employee Name: {self.ename}")
        print(f"Basic Salary: {self.basic}")
        print(f"Dearness Allowance (DA): {self.da}")
        print(f"House Rent Allowance (HRA): {self.hra}")
        print(f"Total Salary: {total_salary}")

# Create two employee instances
emp1 = Employee(101, "John Doe", 50000)
emp2 = Employee(102, "Jane Smith", 60000)

# Display payroll for both employees
print("Payroll Details for Employee 1:")
emp1.display_payroll()
```

```
print("\nPayroll Details for Employee 2:")
emp2.display_payroll()

# Display total employees (should be 2)
print(f"\nTotal Employees: {Employee.total_employees}")

# Deleting one employee
del emp1

# Display total employees after deleting one (should be 1)
print(f"Total Employees after deletion: {Employee.total_employees}")
```

2. Develop a class for student(sno, sname, mark1, mark2, mark3) and display
the result as pass if a student scored above 50% in all the subjectsotherwise
display “FAIL”.

```
class Student:
    def __init__(self, sno, sname, mark1, mark2, mark3):
        self.sno = sno
        self.sname = sname
        self.mark1 = mark1
        self.mark2 = mark2
        self.mark3 = mark3

    def calculate_result(self):
        # Calculate the average mark for the three subjects
        average_mark = (self.mark1 + self.mark2 + self.mark3) / 3

        # Check if the average mark is above 50%
        if average_mark >= 50:
            return "PASS"
        else:
```

```
        return "FAIL"

    def display_result(self):
        result = self.calculate_result()
        print(f"Student Number: {self.sno}")
        print(f"Student Name: {self.sname}")
        print(f"Mark 1: {self.mark1}")
        print(f"Mark 2: {self.mark2}")
        print(f"Mark 3: {self.mark3}")
        print(f"Result: {result}")

# Create a student instance
student1 = Student(1, "John Doe", 75, 60, 80)
student2 = Student(2, "Jane Smith", 40, 55, 30)

# Display the results for both students
print("Result for Student 1:")
student1.display_result()
print("\nResult for Student 2:")
student2.display_result()
```

3. Write a python program to observe the output of various [], .,^,$,*, {} in
findall() & search() methods of regular expression.

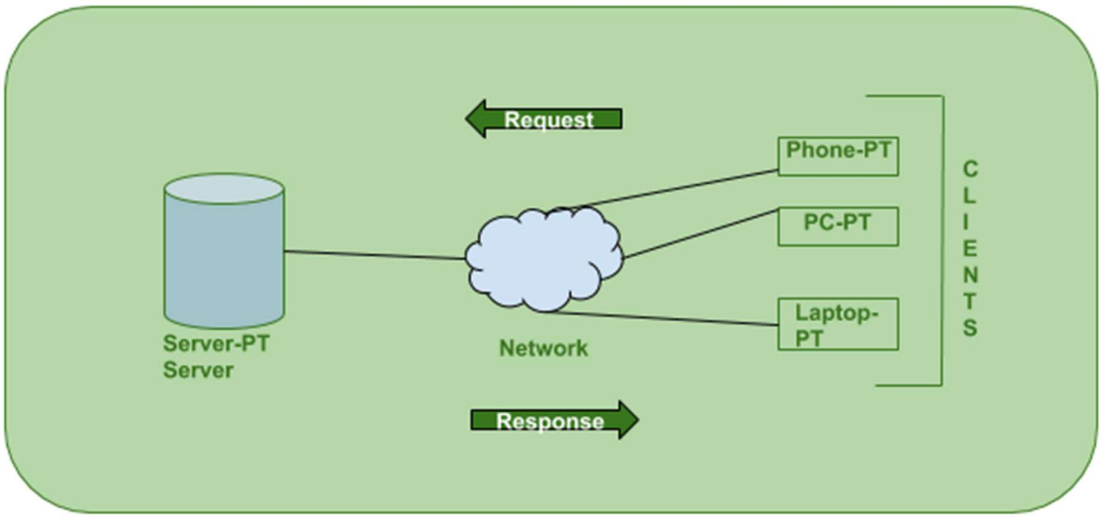
```
import re

# Sample text
text = "The quick brown fox jumps over the lazy dog."
```

```
# Using findall() with []
pattern = "[aeiou]"  # Matches any vowel
result = re.findall(pattern, text)
print("Find all vowels:", result)

# Using findall() with .
pattern = "o."  # Matches 'o' followed by any character
result = re.findall(pattern, text)
print("Find all 'o' followed by any character:", result)

# Using findall() with *
pattern = "o.*"  # Matches 'o' followed by zero or more characters
result = re.findall(pattern, text)
print("Find 'o' followed by zero or more characters:", result)

# Using findall() with {}
pattern = "o.{1,3}"  # Matches 'o' followed by 1 to 3 characters
result = re.findall(pattern, text)
print("Find 'o' followed by 1 to 3 characters:", result)

# Using search() with ^
pattern = "^The"  # Matches 'The' at the beginning of the text
result = re.search(pattern, text)
if result:
    print("Search for 'The' at the beginning:", result.group())
else:
    print("Pattern not found at the beginning")

# Using search() with $
pattern = "dog.$"  # Matches 'dog.' at the end of the text
result = re.search(pattern, text)
if result:
    print("Search for 'dog.' at the end:", result.group())
else:
    print("Pattern not found at the end")
```

UNIT -3

Part-B

1. Describe the client-server architecture of Web applications



The Client-server model is a distributed application structure that partitions task or workload between the providers of a resource or service, called servers, and service requesters called clients.

In the client-server architecture, when the client computer sends a request for data to the server through the internet, the server accepts the requested process and deliver the data packets requested back to the client.

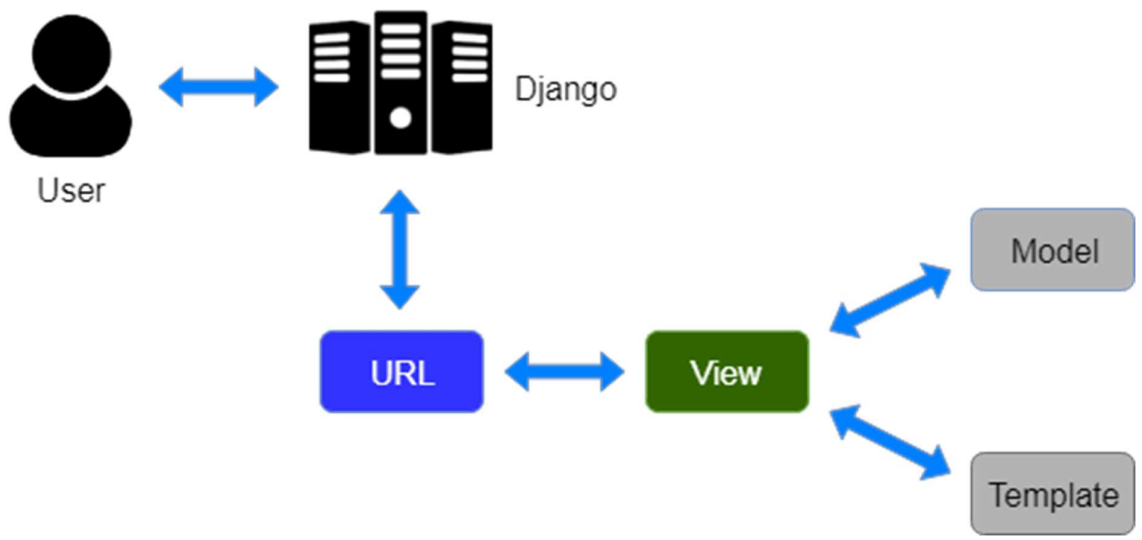
Clients do not share any of their resources.

Examples of Client-Server Model are Email, World Wide Web, etc.

How the Client-Server Model works ?
Client: When we talk the word Client, it mean to talk of a person or an organization using a particular service. Similarly in the digital world a Client is a computer (Host) i.e. capable of receiving information or using a particular service from the service providers (Servers).
Servers: Similarly, when we talk the word Servers, It mean a person or medium that serves something. Similarly in this digital world a Server is a

remote computer which provides information (data) or access to particular services.

2. Elaborate the MVT model of Django with neat sketch

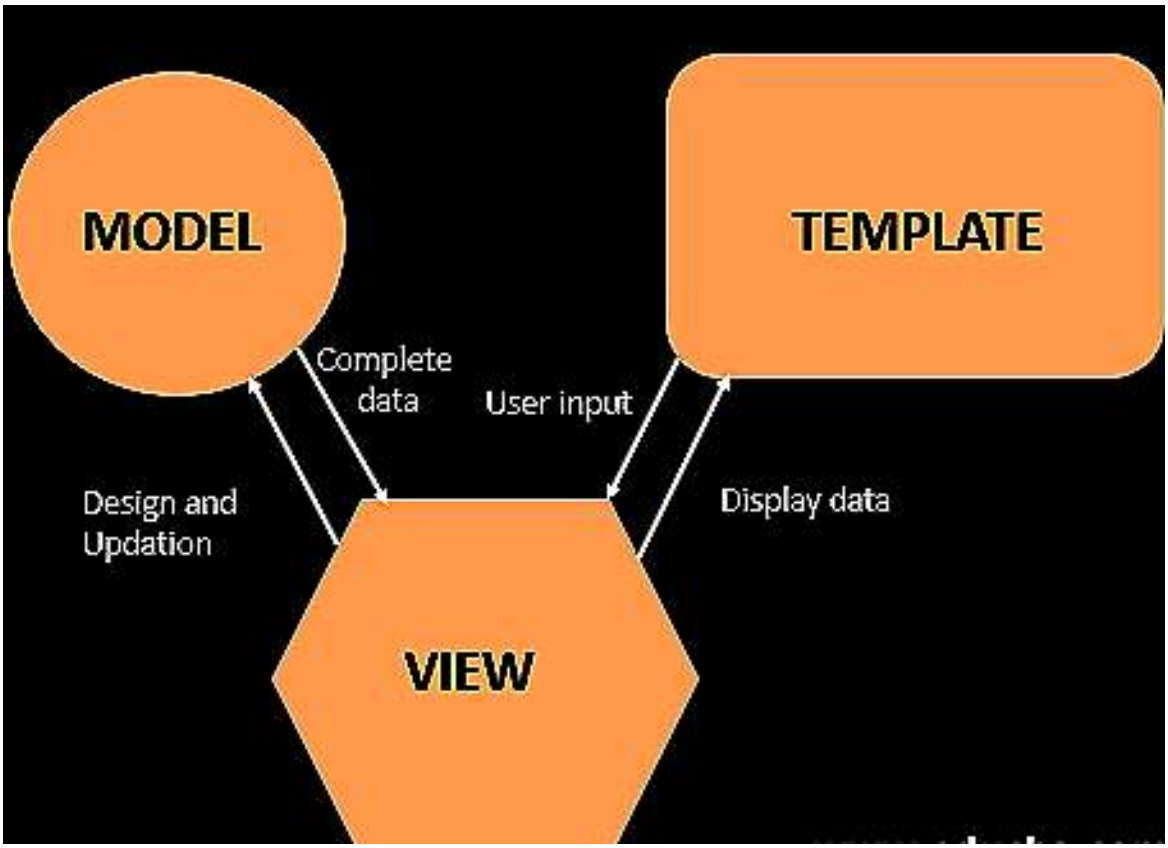


Django is based on MVT (Model-View-Template) architecture.
MVT is a software design pattern for developing a web application.
MVT has three parts:
Model
View
Template

The Model helps to handle database. It is a data access layer which handles the data.
The Template is a presentation layer which handles User Interface part completely. The View is used to execute the business logic and interact with a model to carry data and renders a template.
Although Django follows MVC pattern but maintains it's own conventions. So, control is handled by the framework itself.
There is no separate controller and complete application is based on Model View and Template. That's why it is called MVT application.

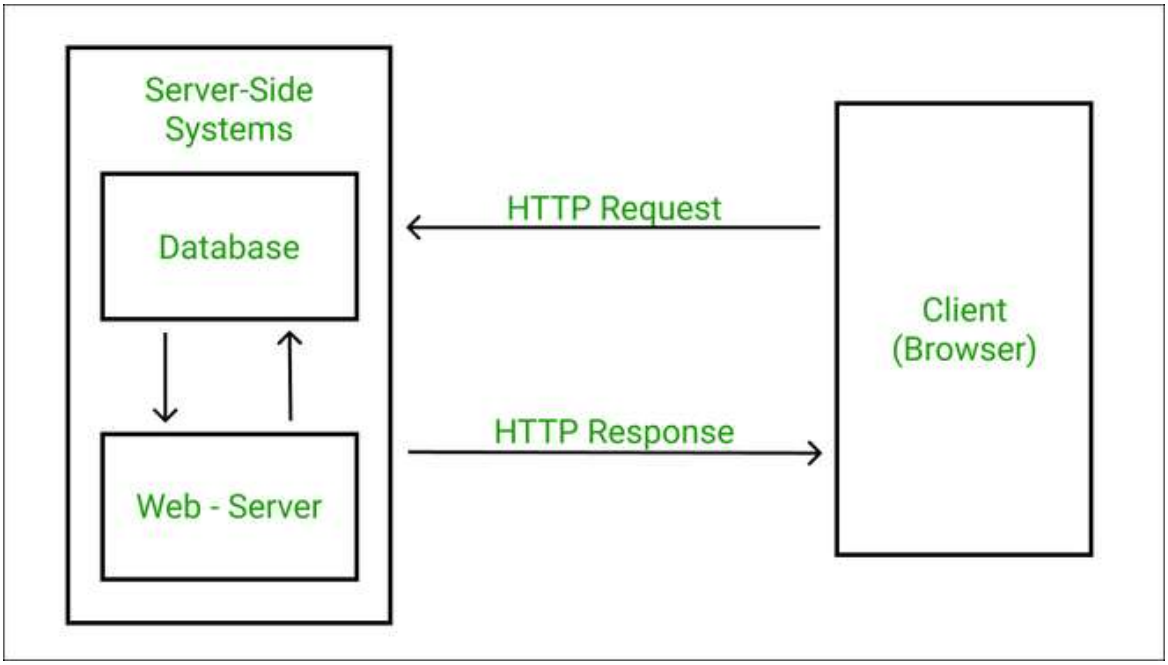
Here, a user requests for a resource to the Django, Django works as a controller and check to the available resource in URL.

If URL maps, a view is called that interact with model and template, it renders a template.
Django responds back to the user and sends a template as a response.



Model: The model is going to act as the interface of your data. It is responsible for maintaining data. It is the logical data structure behind the entire application and is represented by a database (generally relational databases such as MySql, Postgres).
View: The View is the user interface — what you see in your browser when you render a website. It is represented by HTML/CSS/Javascript and Jinja files.
Template: A template consists of static parts of the desired HTML output as well as some special syntax describing how dynamic content will be inserted.

3. Discuss in details about HTTP request and response objects with sample
program



The client-server architecture includes two major components request and response.
The Django framework uses client-server architecture to implement web applications.
When a client requests for a resource, a HttpRequest object is created and correspond view function is called that returns HttpResponse object.
To handle request and response, Django provides HttpRequest and HttpResponse classes.
Each class has it's own attributes and methods.

Django uses request and response objects to pass state through the system.

When a page is requested, Django creates an HttpRequest object that contains metadata about the request. Then Django loads the appropriate view, passing the HttpRequest as the first argument to the view function. Each view is responsible for returning an HttpResponse object.

Django HttpRequest
This class is defined in the django.http module and used to handle the client request.

Django HttpRequest Example

views.py

```
def methodinfo(request):
    return HttpResponse("Http request is: "+request.method)
```

urls.py

```
path('info',views.methodinfo)
```

Django uses request and response objects to pass state through the system. When a page is requested, Django creates an HttpRequest object that contains metadata about the request. Then Django loads the appropriate view, passing the HttpRequest as the first argument to the view function. Each view is responsible for returning an HttpResponse object.

This class is a part of django.http module.
It is responsible for generating response corresponds to the request and back to the client.

4. Explain the process of template loading and rendering in Django with code
segments

Template Loading

Setup Directory: Create a directory named templates within your Django app's directory.

Create HTML Templates: Inside the templates directory, create HTML template files that define the structure of web pages with placeholders for dynamic content.

Load Templates in Views: In your Python view functions, use the render function from django.shortcuts to load templates. Pass the request, template file name (without the file extension), and context data as arguments.

Define Context Data: Create a dictionary called context within your view function to hold data you want to pass to the template.

Render Template: Call render with the request, template name, and context. Django will automatically find the template in the templates directory.

HTTP Response: render returns an HttpResponse object with the rendered HTML. This response is sent to the client's browser when the view is accessed.

Rendering

Create Template Files: Design HTML templates with placeholders for dynamic data.

Define View Function: Create a Python view function that retrieves data and renders the template using the render function.

Configure URLs: Map URLs to view functions in your app's urls.py file.

Render Template: When a user accesses a URL, Django invokes the view, rendering the template with dynamic data.

Display Rendered Page: Users see the dynamically generated web page with content from the template when they access the URL.

5. Describe the process of using Django Models in web application with example.

Define a Django Model:
Create a Python class that subclasses django.db.models.Model to define your database structure (fields).

Create Database Tables:
Run python manage.py makemigrations and python manage.py migrate to create database tables based on your models.

Interact with the Database:
Use the Django ORM to perform CRUD (Create, Read, Update, Delete) operations on your models.

Use Models in Views:
In your views, query the database using models and pass the data to templates for rendering.

Render Data in Templates:
In templates, use template tags and filters to display data retrieved from models.

Configure URLs:
Define URL patterns in your app's urls.py to map URLs to view functions.

Start the Development Server:
Run python manage.py runserver to test your application locally.

This process allows you to define and interact with a database in your Django web application, making it easier to manage and display data.

6. Develop a web application to display “Hello World” and describe the steps.

Step 1: Install Flask
If you haven't already, you need to install Flask. You can do this using pip, the Python package manager:
bash
Copy code
```
pip install Flask
```

Step 2: Create the Flask Application
Create a Python file (e.g., app.py) to define your Flask application. In this file, you'll import Flask, create an instance of the Flask app, and define a route to display "Hello World."

python
Copy code
```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello World'

if __name__ == '__main__':
    app.run()
```

In the code above:
We import Flask from the Flask module.

We create a Flask application instance called app.
We define a route '/' using the @app.route decorator. When a user visits the root URL ('/'), the hello_world function is executed.
The hello_world function returns the string 'Hello World.'
Finally, we start the Flask development server if the script is executed directly (not imported as a module).

Step 3: Run the Flask Application
To run your Flask application, open a terminal in the same directory as your app.py file and execute the following command:

bash
Copy code
python app.py
You'll see output indicating that the Flask development server is running. By default, the server runs on http://localhost:5000.

Step 4: Access the Application in a Web Browser
Open a web browser and navigate to http://localhost:5000 (or the URL displayed in your terminal). You should see the "Hello World" message displayed in your browser.

7. Enumerate the files created while starting a Django Project.

When you start a new Django project using the `django-admin startproject` command, it creates a directory with the project name and several files and directories within it. Here's a list of the primary files and directories created when starting a Django project:

1. **Project Directory:**
   - This is the main directory with the name you specified when running `startproject`. It contains all the project-related files and directories.

2. **`manage.py`:**

- This Python script is used for various administrative tasks related to the Django project, such as running the development server, creating database tables, and making migrations.

3. **Project Settings:**
   - The project directory contains a Python module (usually named after your project) that stores project-specific settings. The main settings file is typically named `settings.py`. This file includes database configuration, middleware settings, installed apps, and more.

4. **`__init__.py`:**
   - This is an empty Python file that indicates the directory should be treated as a Python package.

5. **`asgi.py` and `wsgi.py`:**
   - These are entry points for ASGI (Asynchronous Server Gateway Interface) and WSGI (Web Server Gateway Interface) servers, respectively. They are used for deploying Django applications to different types of web servers.

6. **`urls.py`:**
   - This file contains the URL patterns for your project. It defines the mapping between URLs and view functions, allowing you to route incoming requests to the appropriate views.

7. **`settings` Directory:**
   - Inside the project settings directory, you'll find additional settings files, including `__init__.py`, `base.py`, `development.py`, `production.py`, and `test.py`. These files allow you to organize your settings for different environments (e.g., development, production).

8. **`migrations` Directory:**
   - This directory is created when you run `makemigrations` to manage database schema changes. It stores migration files that describe how the database schema should be updated.

9. **`__pycache__` Directories:**
   - These directories store compiled Python bytecode files and can be ignored.

10. **Virtual Environment Directory (Optional):**
    - If you create a virtual environment for your project (which is a recommended practice), it will contain its own directory with Python packages and libraries specific to your project.

These are the essential files and directories created when starting a Django project. The structure may vary slightly depending on the version of Django you're using, but the core components remain consistent.

PART C

1. Explain the significance of templates in web applications and how templates are
used in the Django framework with example application.

Templates in Web Applications:

Templates play a crucial role in web applications by separating the presentation (the user interface or how content is displayed) from the logic (the underlying code that generates or processes data). Here's the significance of templates in web applications:

1. **Separation of Concerns:** Templates allow developers to separate the HTML markup and visual design (presentation) from the application's business logic and data processing. This separation enhances code maintainability and collaboration among designers and developers.

2. **Reusability:** Templates can be reused across multiple pages or views, promoting consistency in the appearance and layout of a web application. Changes to the template automatically reflect on all pages that use it.

3. **Dynamic Content:** Templates support dynamic content rendering. Variables and control structures within templates enable the display of data retrieved from databases, user input, or other sources.

4. **User Experience:** Templates contribute significantly to the user experience by defining how content is structured and presented. A well-designed template can enhance usability and accessibility.

Templates in Django:

In Django, templates are used to generate HTML and other content dynamically. Django's template system is powerful and flexible, allowing developers to create dynamic web pages effortlessly. Here's how templates are used in Django with an example application:

Step 1: Create a Django Project and App

```bash
django-admin startproject myproject
cd myproject
python manage.py startapp myapp
```

Step 2: Define a Model (models.py)

Create a simple model in `models.py` that represents data. For example:

```python
from django.db import models

class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()

    def __str__(self):
        return self.title
```

Step 3: Create a View (views.py)

Define a view function in `views.py` to retrieve data and render it using a template:

```python
from django.shortcuts import render
from .models import Post

def post_list(request):
    posts = Post.objects.all()
    return render(request, 'myapp/post_list.html', {'posts': posts})
```

Step 4: Create a Template (post_list.html)

Create an HTML template file (`post_list.html`) within the app's `templates` directory to render the data:

```html
<!DOCTYPE html>
<html>
<head>
```

```
    <title>Post List</title>
</head>
<body>
    <h1>Blog Posts</h1>
    <ul>
        {% for post in posts %}
            <li>{{ post.title }}</li>
            <p>{{ post.content }}</p>
        {% endfor %}
    </ul>
</body>
</html>
```

Step 5: Configure URLs (urls.py)

Define a URL pattern in the app's `urls.py` to map the URL to the view function:

```python
from django.urls import path
from . import views

urlpatterns = [
    path('posts/', views.post_list, name='post_list'),
]
```

Step 6: Start the Development Server

Run the development server:

```bash
python manage.py runserver
```

Visit http://localhost:8000/posts/ to see the list of blog posts rendered using the template.

In this example, the Django template (`post_list.html`) separates the HTML presentation from the view logic, allowing dynamic rendering of data from the database. This separation enhances code organization and maintainability while delivering a dynamic user experience.

2. Develop a web application using Django framework to validate username (not

blank, max 20 characters), review_comment (not null) & gender (choices =
{male,female}, not null) using forms. Describe the steps.

To develop a web application using the Django framework to validate fields like `username`, `review_comment`, and `gender` using forms, you can follow these steps:

Step 1: Create a Django Project and App

If you haven't already, create a new Django project and an app within the project. You can do this using the following commands:

```bash
django-admin startproject myproject
cd myproject
python manage.py startapp myapp
```

Step 2: Define the Model

In your app's `models.py` file, define a model that represents the data you want to collect. In this case, you may define a model for user reviews:

```python
from django.db import models

class UserReview(models.Model):
    username = models.CharField(max_length=20)
    review_comment = models.TextField()
    GENDER_CHOICES = [
        ('male', 'Male'),
        ('female', 'Female'),
    ]
    gender = models.CharField(max_length=6, choices=GENDER_CHOICES)

    def __str__(self):
        return self.username
```

Step 3: Create a Form

Django provides a `ModelForm` class that can automatically generate a form based on your model. Create a form for the `UserReview` model in your app's `forms.py` file:

```python
from django import forms
from .models import UserReview

class UserReviewForm(forms.ModelForm):
    class Meta:
        model = UserReview
        fields = ['username', 'review_comment', 'gender']
```

Step 4: Create a View

In your app's `views.py` file, create a view function that handles the form submission and rendering of the form:

```python
from django.shortcuts import render
from .forms import UserReviewForm

def review_form(request):
    if request.method == 'POST':
        form = UserReviewForm(request.POST)
        if form.is_valid():
            form.save()
    else:
        form = UserReviewForm()
    return render(request, 'myapp/review_form.html', {'form': form})
```

Step 5: Create a Template for the Form

Create an HTML template (`review_form.html`) within your app's `templates` directory to render the form:

```html
<!DOCTYPE html>
<html>
<head>
    <title>User Review Form</title>
</head>
<body>
```

```
    <h1>User Review Form</h1>
    <form method="post">
        {% csrf_token %}
        {{ form.as_p }}
        <button type="submit">Submit</button>
    </form>
</body>
</html>
```

Step 6: Configure URLs

Define a URL pattern in your app's `urls.py` to map the URL to the view function:

```python
from django.urls import path
from . import views

urlpatterns = [
    path('review/', views.review_form, name='review_form'),
]
```

Step 7: Run the Development Server

Run the development server:

```bash
python manage.py runserver
```

Visit http://localhost:8000/review/ to access the user review form. The form will validate the `username`, `review_comment`, and `gender` fields as per the model and form definitions. Users can submit their reviews, and valid submissions will be saved in the database.

This example demonstrates how to create a Django web application that validates form fields and stores the data in the database. You can further customize the form and validation rules as needed for your application.

UNIT-4

PART B

1. List the significant features of the pandas Library and Demonstrate the arithmetic operations on data frames with code segments

The pandas library in Python is a powerful tool for data manipulation and analysis. Some of its significant features include:

Data Structures: It provides two main data structures, Series and DataFrame, which are designed for handling labelled and relational data efficiently.

Data Handling: Pandas allows for reading and writing data from various file formats such as CSV, Excel, SQL databases, JSON, etc.

Data Alignment: It aligns data automatically in arithmetic operations based on the labelled axes, making calculations more straightforward and intuitive.

Missing Data Handling: Pandas provides functionalities to handle missing data using methods like dropna(), fillna(), and isnull().

Grouping and Aggregation: It enables grouping data and performing operations on groups of data using functionalities like groupby().

Time Series Analysis: Pandas offers powerful tools to work with time series data, including date range generation, shifting, lagging, and frequency conversion of time series.

Here's an example demonstrating arithmetic operations on DataFrames in pandas:

```
import pandas as pd
# Creating two sample DataFrames
data1 = {
'A': [1, 2, 3],
'B': [4, 5, 6],
'C': [7, 8, 9]
}
df1 = pd.DataFrame(data1)

data2 = {
'A': [10, 20, 30],
'B': [40, 50, 60],
'C': [70, 80, 90]
}
df2 = pd.DataFrame(data2)

# Adding two DataFrames
result_sum = df1 + df2
print("Addition:")
print(result_sum)
```

```
print()

# Subtracting two DataFrames
result_diff = df1 - df2
print("Subtraction:")
print(result_diff)
print()

# Multiplying two DataFrames
result_mul = df1 * df2
print("Multiplication:")
print(result_mul)
print()

# Dividing two DataFrames
result_div = df1 / df2
print("Division:")
print(result_div)
```

This code creates two DataFrames (df1 and df2) and performs basic arithmetic operations such as addition, subtraction, multiplication, and division on these DataFrames. The results will show the outcome of these operations, element-wise, preserving the DataFrame structure.

2. Elaborate the problem faced in handling large data & describe the programming tips to handle large data with examples.

Handling large data poses several challenges, such as memory constraints, slower processing times, and inefficiencies in data manipulation. Here are some common issues faced when working with large datasets:

1. **Memory Constraints:** Traditional data structures may not fit into memory, leading to slowdowns or crashes.
2. **Processing Time:** Operations on large datasets take longer, impacting the efficiency of data analysis and manipulation.
3. **I/O Operations:** Reading and writing large files can be slow, causing delays in data processing.
4. **Data Cleaning:** Preprocessing and cleaning large datasets might require complex operations, which can be time-consuming.

Here are some programming tips and techniques to handle large datasets more efficiently:

1. **Use Chunking for Reading Data:** When working with large files, use chunking to read the data in smaller portions. Pandas allows you to read large files in chunks using the `chunksize` parameter in `pd.read_csv()`. Here's an example:

```
chunk_size = 10000 # Define the chunk size
for chunk in pd.read_csv('large_file.csv', chunksize=chunk_size):
# Process each chunk (e.g., perform operations or store it)
process_data(chunk)
```

2. **Optimize Data Types:** Use appropriate data types to reduce memory usage. For example, use `int32` instead of `int64` or `float32` instead of `float64` if the data range allows. This can significantly reduce memory usage.

```
# Convert data types to save memory
df['column_name'] = df['column_name'].astype('int32')
```

3. **Filtering and Querying:** Perform filtering and querying operations directly while reading the data to extract only the necessary information. This reduces the amount of data that needs to be loaded into memory.

```
# Read data where column 'A' values are greater than 100
filtered_data = pd.read_csv('large_file.csv', usecols=['A'],
chunksize=chunk_size)
```

```
for chunk in filtered_data:
filtered_chunk = chunk[chunk['A'] > 100]
process_data(filtered_chunk)
```

4. **Use Dask for Parallel Processing:** Dask is a parallel computing library that integrates with Pandas and NumPy and can handle larger-than-memory datasets by breaking them into smaller tasks. It provides parallel computation for Pandas-like operations.

```
import dask.dataframe as dd

# Read data using Dask
df = dd.read_csv('large_file.csv')

# Perform operations on Dask DataFrame
result = df[df['column_name'] > 100].compute()
```

5. **Consider Database Systems:** For extremely large datasets, consider using database systems like SQL or NoSQL databases to efficiently query and manipulate data without needing to load the entire dataset into memory.

Efficiently handling large datasets involves a combination of these techniques tailored to the specific requirements of the dataset and the analysis being performed.

3. Develop a python program to perform aggregate functions of pivoted tables.

Dataset

Name	Gender	Age
John	Male	45
Sammy	Female	6
Stephan	Male	4
Joe	Female	36
Emily	Female	12
Tom	Male	43

Pivot Table

Gender	%Gender	Age Group	Count
Male	50%	>18 years	2
		<18 years	1
Female	50%	>18 years	1
		<18 years	2

Certainly! To perform aggregate functions on pivoted tables, you can use the pandas library in Python. Here's an example demonstrating how to create a pivoted table from a sample dataset and perform aggregate functions on it:

```
import pandas as pd

# Sample dataset
data = {
'Date': ['2023-01-01', '2023-01-01', '2023-01-02', '2023-01-02'],
'Category': ['A', 'B', 'A', 'B'],
'Value': [10, 20, 15, 25]
}

# Create a DataFrame from the sample dataset
df = pd.DataFrame(data)

# Pivot the DataFrame
pivot_table = df.pivot_table(index='Date', columns='Category',
values='Value', aggfunc='sum')

print("Pivoted Table:")
print(pivot_table)
print()

# Perform aggregate functions on the pivoted table
# For example, calculating the mean across columns
mean_across_columns = pivot_table.mean()
print("Mean across columns:")
print(mean_across_columns)
print()
```

```
# You can perform various aggregate functions like sum, mean, min, max, etc. as per your requirement.
# For instance, calculating the sum across rows
sum_across_rows = pivot_table.sum(axis=1)
print("Sum across rows:")
print(sum_across_rows)
```

This program uses a sample dataset, creates a DataFrame, pivots the table based on 'Date' and 'Category', and then demonstrates aggregate functions such as mean across columns and sum across rows using the pivot_table and pandas aggregate functions (mean() and sum() in this case).

Modify the aggfunc parameter in pivot_table according to the specific aggregation function (e.g., 'sum', 'mean', 'min', 'max') you want to apply to the pivoted table. Also, adjust the index and columns based on your dataset to perform aggregate functions across rows or columns in the pivoted table.

4. Write a Python program to observe the relationship between variables using crosstab.

Certainly! The crosstab function in pandas is used to compute a cross-tabulation table that can show the relationship between variables. Here's an example Python program that demonstrates how to use crosstab to observe the relationship between variables in a dataset:

```
import pandas as pd

# Sample dataset
data = {
'Category': ['A', 'B', 'A', 'B', 'A', 'B', 'A', 'A'],
'Value1': [10, 20, 15, 25, 18, 30, 12, 8],
'Value2': [35, 22, 18, 45, 27, 32, 40, 15]
}

# Create a DataFrame from the sample dataset
df = pd.DataFrame(data)

# Using crosstab to observe the relationship between 'Category' and
'Value1'
cross_tab = pd.crosstab(df['Category'], df['Value1'])
print("Cross-tabulation table:")
print(cross_tab)
```

This program generates a sample dataset, creates a DataFrame, and then uses the pd.crosstab() function to observe the relationship between the 'Category' column and 'Value1'. The output will display a cross-tabulation table where you can see the frequency of occurrences of each 'Value1' for each 'Category'.

You can modify the columns used in crosstab according to your dataset to explore the relationship between different variables. This method allows you to understand the distribution and relationship between categorical variables and can be extended to explore relationships between multiple columns in more complex datasets.

5. Develop python program to perform timedelta operation on time-series data

Certainly! Performing timedelta operations on time-series data can involve various operations such as adding or subtracting time intervals, finding time differences, etc. This can be done using the timedelta functionality available in Python's datetime module. Here's an example Python program that demonstrates timedelta operations on time-series data:

```
from datetime import datetime, timedelta
import pandas as pd

# Create a sample time-series dataset
dates = pd.date_range(start='2023-11-01', periods=5, freq='D')
data = {'Values': [10, 20, 15, 30, 25]}
df = pd.DataFrame(data, index=dates)
print("Original Time-Series Data:")
print(df)
print()

# Adding a fixed time interval to the entire time series
time_interval_to_add = timedelta(days=3)
df.index = df.index + time_interval_to_add
print("Time-Series Data after adding a time interval of 3 days:")
print(df)
print()

# Calculating time differences between consecutive timestamps
df['Time_Difference'] = df.index.to_series().diff()
print("Time Differences between Consecutive Timestamps:")
print(df)
```

Explanation of the program:

Creating a Sample Time-Series Dataset:

The program creates a sample time-series dataset using pd.date_range and generates random data.
Adding a Fixed Time Interval:

It adds a fixed time interval (3 days in this case) to the entire time series using timedelta arithmetic. This shifts the entire index by three days.
Calculating Time Differences:

It calculates the time differences between consecutive timestamps and creates a new column 'Time_Difference' in the DataFrame.

The output will display the original time-series data, the time-series data after adding the time interval, and the calculated time differences between consecutive timestamps.

You can perform various other operations like subtracting time intervals, finding the difference between specific timestamps, etc., using timedelta operations on time-series data. This enables manipulation and analysis of time-based data by leveraging the datetime and timedelta functionalities in Python.

6. Explain indexing and multi-indexing data frames with code segments.

In Pandas, indexing refers to accessing specific elements or subsets of a DataFrame using various methods. DataFrames can be indexed in different ways, allowing for flexibility and ease in data manipulation. Multi-indexing, also known as hierarchical indexing, involves creating a DataFrame with multiple levels of index, allowing for more complex data organisation and retrieval.

Indexing DataFrames:
Using .loc[] and .iloc[]:.loc[] is label-based indexing, allowing you to access data using column labels and row indices.
.iloc[] is integer-based indexing, which lets you access data using integer-based positions of rows and columns.
pythonCopy code
```
import pandas as pd

# Sample DataFrame
data = {
'A': [1, 2, 3],
'B': [4, 5, 6],
'C': [7, 8, 9]
}
df = pd.DataFrame(data)

# Label-based indexing using .loc[]
print(df.loc[0, 'B']) # Accessing the value in row 0 and column 'B'

# Integer-based indexing using .iloc[]
print(df.iloc[1, 2]) # Accessing the value in row 1 and column 2
```

Conditional Indexing:You can use conditions to filter rows or select specific elements in a DataFrame.
pythonCopy code
```
# Conditional indexing
filtered_df = df[df['A'] > 1] # Selecting rows where column 'A' > 1
print(filtered_df)
```

Multi-indexing DataFrames:
Creating a DataFrame with a multi-index involves specifying multiple levels of index using pd.MultiIndex.

pythonCopy code
```
# Creating a multi-index DataFrame
arrays = [
['A', 'A', 'B', 'B'],
[1, 2, 1, 2]
]
multi_index = pd.MultiIndex.from_arrays(arrays, names=('First', 'Second'))

data = {
'Values': [11, 22, 33, 44],
'Values2': [55, 66, 77, 88]
}
multi_df = pd.DataFrame(data, index=multi_index)

print(multi_df)
```

Accessing elements in a multi-index DataFrame can be done using loc[] with tuples representing the multiple levels.

pythonCopy code
```
# Accessing values in a multi-index DataFrame
print(multi_df.loc['A']) # Accessing values under the first index 'A'

print(multi_df.loc[('A', 1), 'Values']) # Accessing specific value under ('A', 1) in 'Values' column
```

Multi-indexing is very powerful when dealing with more complex, hierarchical data, as it allows for more structured and meaningful organization of information in the DataFrame.

7. Discuss about the categorical data with examples.

Categorical data refers to a type of data that represents categories or groups. It's used to label variables that have a finite and fixed number of possible values. Categorical data can be divided into nominal and ordinal categories.

Nominal Categorical Data:
Nominal data represents categories with no inherent order or ranking.

Example 1: Categorical Data as Nominal
Consider a dataset containing information about different fruits:

Fruit	Color	Taste
Apple	Red	Sweet
Banana	Yellow	Sweet
Orange	Orange	Tangy
Grape	Purple	Sweet

In this dataset, 'Fruit', 'Color', and 'Taste' are categorical variables. 'Fruit' is a nominal categorical variable as there is no inherent order among different fruits.

Ordinal Categorical Data:
Ordinal data represents categories with a specific order or ranking.

Example 2: Categorical Data as Ordinal
Consider a dataset with student grades:

Student	Grade
John	A
Lisa	B
Mike	C
Sarah	A

In this case, 'Grade' is an ordinal categorical variable because the grades have an inherent order (A > B > C).

Using Categorical Data in Pandas:

In Pandas, categorical data type can be used to improve performance and memory usage when working with large datasets. You can convert a column to a categorical type using `pd.Categorical()` or `astype('category')`.

```python
import pandas as pd

# Creating a DataFrame
data = {'Fruit': ['Apple', 'Banana', 'Orange', 'Grape'],
        'Color': ['Red', 'Yellow', 'Orange', 'Purple'],
        'Taste': ['Sweet', 'Sweet', 'Tangy', 'Sweet']}
df = pd.DataFrame(data)

# Converting 'Fruit' column to categorical type
df['Fruit'] = pd.Categorical(df['Fruit'])

# Display the data types
print(df.dtypes)
```

Benefits of using categorical data:
1. **Memory Efficiency:** Categorical data is represented by integers, so it consumes less memory.
2. **Performance Improvement:** Many operations are faster on categorical data because they are internally stored as integers.

Categorical data provides an efficient way to work with non-numeric data, enabling easier analysis and manipulation in data science tasks.

8. How would you iterate over rows in a DataFrame in Pandas?

In Pandas, you can iterate over rows in a DataFrame using various methods. Here are some common approaches:

Using `iterrows()` method:

The `iterrows()` method allows you to iterate through the rows of a DataFrame, returning an index for each row and a Series containing the data in each row.

```python
import pandas as pd

# Sample DataFrame
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'City': ['New York', 'San Francisco', 'Los Angeles']
}
df = pd.DataFrame(data)

# Iterating over rows using iterrows()
for index, row in df.iterrows():
    print(f"Index: {index}")
    print(row)
```

Using a `for` loop with `iterrows()`:

You can access specific columns' values within the loop.

```
for index, row in df.iterrows():
    print(f"Index: {index}")
    print(f"Name: {row['Name']}, Age: {row['Age']}, City: {row['City']}")
```

Note:
While `iterrows()` is convenient, it might not be the most efficient way to handle large DataFrames due to its slower performance. For large datasets, vectorized operations or other methods like `apply()` or list comprehensions might be more efficient.

Using `apply()` method:

Another efficient way to perform operations on rows is by using the `apply()` method along the axis.

```python
# Applying a function to each row
def process_row(row):
    # Perform operations on each row
    return row['Age'] * 2

# Applying the function to each row using apply()
doubled_age = df.apply(process_row, axis=1)
print(doubled_age)
```

This method applies the function `process_row` to each row of the DataFrame, performing the specified operations efficiently.

Each method for iterating over rows in a DataFrame has its own advantages and use cases. Consider the size of your dataset and the operations you need to perform when choosing the appropriate method for iterating over rows.

9. List some statistical functions in Python Pandas?
Pandas provides a wide range of statistical functions that are incredibly useful for analyzing and summarizing data in DataFrames. Some of the key statistical functions available in Pandas include:

1. **Descriptive Statistics Functions:**
   - `mean()`: Calculate the mean of numeric columns.
   - `median()`: Compute the median of numeric columns.
   - `mode()`: Find the mode of the values in a column.
   - `std()`: Calculate the standard deviation of numeric columns.
   - `var()`: Compute the variance of numeric columns.
   - `sum()`: Calculate the sum of values.
   - `count()`: Count non-NA/null observations.

2. **Quantile-based Functions:**
   - `quantile()`: Compute sample quantiles.
   - `cumsum()`: Calculate the cumulative sum of a column's values.
   - `cumprod()`: Compute the cumulative product of a column's values.
   - `cummax()` and `cummin()`: Calculate cumulative maximum and minimum, respectively.

3. **Summary Statistics Functions:**
   - `describe()`: Generates descriptive statistics, providing a summary of the central tendency, dispersion, and shape of the dataset's distribution.
   - `info()`: Display a concise summary of the DataFrame, including the data types and non-null values in each column.
   - `value_counts()`: Count unique values in a Series.

4. **Correlation and Covariance Functions:**
   - `corr()`: Compute the correlation matrix between numeric columns.
   - `cov()`: Calculate the covariance matrix between numeric columns.

5. **Resampling Functions:**
   - `resample()`: Resample time-series data, allowing operations over different time intervals.
   - `asfreq()`: Convert time-series to a specified frequency.

6. **Applying Functions to DataFrames:**
   - `apply()`: Apply a function along the axis of a DataFrame.
   - `applymap()`: Apply a function element-wise across the entire DataFrame.
   - `pipe()`: Allows you to use multiple functions in a chain.

Here's an example demonstrating some of these statistical functions:

```python
import pandas as pd
```

```
import numpy as np

# Create a sample DataFrame
data = {
    'A': np.random.randint(0, 100, 5),
    'B': np.random.randn(5),
    'C': ['X', 'Y', 'Z', 'X', 'Y']
}
df = pd.DataFrame(data)

# Descriptive statistics
print(df.describe())  # Summary statistics
print(df['A'].mean())  # Mean of column 'A'
print(df['B'].std())  # Standard deviation of column 'B'

# Correlation and covariance
print(df.corr())  # Correlation matrix
print(df.cov())  # Covariance matrix

# Value counts
print(df['C'].value_counts())  # Count of unique values in column 'C'
```

These functions offer a comprehensive toolbox for conducting statistical analysis and gaining insights from your data using Pandas in Python.

10. What’s the difference between pivot_table() and groupby()?

Both `pivot_table()` and `groupby()` are powerful methods in Pandas used for data aggregation and summarization, particularly in scenarios involving relational databases or spreadsheets. While they serve similar purposes, there are differences in their functionalities:

`pivot_table()`:

- **Reshaping Data**: `pivot_table()` is primarily used for reshaping and summarizing data, pivoting the DataFrame based on the given columns.
- **Aggregation with Multiple Functions**: It allows aggregating values using various functions (e.g., mean, sum, count) for specific rows and columns.
- **Handling Missing Values**: Provides options to handle missing data through the `fill_value` parameter.
- **Multilevel Indexing**: It supports creating a hierarchical index for multidimensional data representation.

Example of `pivot_table()`:

```python
import pandas as pd

# Sample DataFrame
data = {
    'A': ['foo', 'foo', 'foo', 'bar', 'bar', 'bar'],
    'B': ['one', 'one', 'two', 'two', 'one', 'one'],
    'C': [1, 2, 3, 4, 5, 6]
}
df = pd.DataFrame(data)

# Pivot table example
pivot = df.pivot_table(values='C', index='A', columns='B', aggfunc='sum')
print(pivot)
```

`groupby()`:

- **Split-Apply-Combine Approach**: `groupby()` splits the DataFrame into groups based on a given key and then applies an aggregate function on each group.
- **Flexible Aggregation**: It allows applying various aggregation functions on different columns within the same group.
- **Flexible Indexing**: The resulting object can be a Series or DataFrame.
- **Better for Complex Operations**: `groupby()` is more flexible for complex aggregations or custom functions on groups.

Example of `groupby()`:

```python
# Groupby example
grouped = df.groupby(['A', 'B'])['C'].sum()
print(grouped)
```

Key Differences:

- `pivot_table()` is more focused on reshaping and summarizing the data by pivoting, allowing you to create a new table derived from the original DataFrame.
- `groupby()` is more focused on the split-apply-combine approach, where data is split into groups and operations are applied to each group, then the results are combined.

In summary, `pivot_table()` is better for reshaping and summarizing data by pivoting it, whereas `groupby()` is more flexible and powerful for applying operations on grouped data. The choice between these methods depends on the specific analysis and transformation requirements for your data.

Groupby:

df

	foo	bar	baz	zoo
0	one	A	1	x
1	one	A	2	y
2	one	B	3	z
3	two	A	4	q
4	two	B	5	w
5	two	B	6	t

df.groupby(by=['foo', 'bar'])['baz'].sum()

foo	bar	
one	A	1 + 2
	B	3
two	A	4
	B	5 + 6

Pivot table:

df

	foo	bar	baz	zoo
0	one	A	1	x
1	one	A	2	y
2	one	B	3	z
3	two	A	4	q
4	two	B	5	w
5	two	B	6	t

```
df.pivot_table (index='foo',
                columns='bar',
                values='baz'
                aggfunc='sum') '
```

bar	A	B
foo		
one	1 + 2	3
two	4	5 + 6

11. What’s the difference between interpolate() and fillna() in Pandas?

`interpolate()` and `fillna()` are both methods in Pandas used to manage and handle missing or NaN (Not a Number) values within a DataFrame. However, they differ in their functionality and the way they treat missing data.

`interpolate()` Method:

- **Interpolation of Missing Values**: `interpolate()` is used specifically for filling missing values by interpolating between existing values.
- **Numerical Data Handling**: It is most suitable for numerical data, especially in time series or ordered data, where filling missing values by linear interpolation, polynomial interpolation, etc., could be beneficial.
- **Time-Series Data Handling**: It's often used for filling missing values in time series data to estimate or infer the missing values based on the existing sequence.

Example of `interpolate()`:

```python
import pandas as pd
import numpy as np

# Sample DataFrame with NaN values
data = {'A': [1, np.nan, 3, np.nan, 5]}
df = pd.DataFrame(data)

# Using interpolate to fill NaN values
df['A_interpolated'] = df['A'].interpolate()
print(df)
```

`fillna()` Method:

- **General Missing Value Handling**: `fillna()` is a more general method used to replace NaN values with a specified value or using various filling strategies.
- **Can Fill with Constant Values**: It allows filling NaN values with constants, like 0 or any predefined value, or using methods like 'ffill' (forward fill) or 'bfill' (backward fill).
- **Applicable to Various Data Types**: `fillna()` can be used for both numerical and non-numeric data types.

Example of `fillna()`:

```python
```

```
# Using fillna to fill NaN values with a constant
df['A_filled'] = df['A'].fillna(0)
print(df)
```

Key Differences:

- `interpolate()` is specialized for interpolating missing values, primarily for numerical and ordered data.
- `fillna()` is a more general method that can be used for various types of missing value replacement, allowing filling with specific values or methods regardless of data type.

In summary, `interpolate()` is suitable for numerical data when the missing values are to be estimated by interpolation, whereas `fillna()` is a more versatile method applicable to various data types for filling missing values using different strategies or specific constant values. The choice between the two methods depends on the nature of the missing data and the intended way of handling those missing values.

12. How do you split a DataFrame according to a boolean criterion?

You can split a DataFrame into multiple parts based on a boolean criterion using the `groupby()` method in Pandas. By specifying a boolean condition, you can create subsets or groups within the DataFrame. This method is useful for splitting the DataFrame based on a condition defined by a boolean series.

Here's an example to demonstrate how to split a DataFrame according to a boolean criterion:

```
import pandas as pd

# Sample DataFrame
data = {
    'A': [1, 2, 3, 4, 5],
    'B': [10, 20, 30, 40, 50],
    'C': ['X', 'Y', 'X', 'Y', 'Z']
}
df = pd.DataFrame(data)

# Define a boolean condition (criterion)
condition = df['C'] == 'X'  # Split based on column 'C' values being 'X'

# Split the DataFrame based on the boolean condition using groupby
grouped = df.groupby(condition)

# Access groups
for key, group in grouped:
    print(f"Group: {key}")
    print(group)
```

Output:

```
Group: False
   A   B  C
1  2  20  Y
3  4  40  Y
4  5  50  Z
Group: True
   A   B  C
0  1  10  X
2  3  30  X
```

This code first creates a boolean condition by checking if the values in column 'C' are equal to 'X'. Then, the `groupby()` method is used to split the DataFrame into groups based on this condition. Finally, a loop iterates through the groups created, printing each group.

You can modify the condition to suit different boolean criteria based on the columns and values in your DataFrame. The `groupby()` method is flexible and allows you to split the DataFrame into distinct groups according to various boolean criteria, enabling you to perform different operations or analyses on these subsets of data.

13. How to get frequency count of unique items in a Pandas DataFrame?

In Pandas, you can use the `value_counts()` method to obtain the frequency count of unique items in a DataFrame column. This method is specifically used with Series in Pandas to count the occurrences of each unique value in a column.

Here's an example of how to use `value_counts()` to get the frequency count of unique items in a Pandas DataFrame:

```python
import pandas as pd

# Sample DataFrame
data = {
    'Category': ['A', 'B', 'A', 'C', 'A', 'B', 'B', 'C']
}
df = pd.DataFrame(data)

# Get frequency count of unique items in the 'Category' column
frequency_count = df['Category'].value_counts()

print(frequency_count)
```

This code snippet creates a DataFrame with a column 'Category' and then uses `value_counts()` on the 'Category' column to get the count of unique items along with their frequencies. The output will show the counts of each unique item in the 'Category' column.

You can replace 'Category' with the specific column name you want to obtain the frequency count for within your DataFrame. This method is useful for understanding the distribution of different categories or values within a particular column in your dataset.

14. Describe the computational tools for statistical operations, chart and table visualization in Pandas.

Pandas, along with its integration with other libraries like NumPy, Matplotlib, and Seaborn, provides various computational tools for statistical operations, as well as chart and table visualizations. Here's an overview of some of these tools:

Statistical Operations in Pandas:

1. Descriptive Statistics Functions: Pandas offers a wide range of statistical functions to compute descriptive statistics such as `mean()`, `median()`, `std()`, `var()`, `min()`, `max()`, `sum()`, and more.

2. GroupBy Operations: Using the `groupby()` function, you can group data and perform various operations, including aggregation, transformation, and filtering, on grouped data.

3. Reshaping and Pivoting Data: `pivot_table()` and `melt()` functions help in reshaping data, while `stack()` and `unstack()` are useful for pivoting operations.

Chart Visualization using Pandas:

1. **Integration with Matplotlib**: Pandas integrates well with Matplotlib, a popular plotting library. You can directly call `plot()` on a DataFrame or Series, which uses Matplotlib for generating various types of plots like line plots, bar plots, histograms, scatter plots, etc.

2. **Seaborn Integration**: Seaborn is another visualization library that can be used with Pandas. Seaborn provides additional functionalities for statistical data visualization, offering more complex and aesthetically pleasing plots.

Table Visualization and Manipulation:

1. **Displaying DataFrames**: Pandas provides different display options to visualize DataFrames in various formats (HTML, LaTeX, etc.), and you can control the number of rows/columns displayed with `pd.set_option()`.

2. **Styling DataFrames**: `Styler` in Pandas allows customizing the appearance of DataFrames with colors, bar charts, and other visual elements.

Example:

```python
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Creating a sample DataFrame
data = {
    'A': np.random.randn(100),
    'B': np.random.rand(100),
    'Category': np.random.choice(['X', 'Y', 'Z'], 100)
}
df = pd.DataFrame(data)

# Descriptive statistics
print(df.describe())

# Plotting
df['A'].plot(kind='hist')
plt.show()

# Seaborn integration
sns.scatterplot(x='A', y='B', hue='Category', data=df)
plt.show()

# Table visualization
display(df.head())  # Display the first few rows of the DataFrame
```

This example showcases the use of Pandas for statistical operations, chart visualizations using Matplotlib and Seaborn, and basic table visualization. Combining Pandas with these libraries offers a wide range of tools for data analysis, manipulation, and visualization.

PART C

1. Demonstrate different ways of creating dataframes i. Empty Dataframe ii. dataframe using List iii. DataFrame from dict iv. dataframe from lists using a dictionary iv. dataframe from series

Certainly! Here are examples of different ways to create DataFrames in Pandas:

i. Creating an Empty DataFrame:

You can create an empty DataFrame and later append data to it.

```python
import pandas as pd

# Create an empty DataFrame
empty_df = pd.DataFrame(columns=['A', 'B', 'C'])
print(empty_df)
```

ii. Creating a DataFrame using Lists:

Creating a DataFrame from a list involves using lists as the data source for the DataFrame.

```python
# Creating DataFrame using lists
data_list = [
    [1, 'Alice', 25],
    [2, 'Bob', 30],
    [3, 'Charlie', 35]
]
columns = ['ID', 'Name', 'Age']
df_from_list = pd.DataFrame(data_list, columns=columns)
print(df_from_list)
```

iii. Creating a DataFrame from a Dictionary:

Creating a DataFrame from a dictionary involves keys as column names and values as data.

```python
# Creating DataFrame from a dictionary
```

```python
data_dict = {
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
}
df_from_dict = pd.DataFrame(data_dict)
print(df_from_dict)
```

iv. Creating a DataFrame from Lists using a Dictionary:

This method involves creating a DataFrame from lists using a dictionary to specify column names and data.

```python
# Creating DataFrame from lists using a dictionary
data = {
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
}
columns = list(data.keys())
values = list(data.values())
df_from_lists_dict = pd.DataFrame(dict(zip(columns, values)))
print(df_from_lists_dict)
```

v. Creating a DataFrame from Series:

You can create a DataFrame from Pandas Series, which allows you to combine multiple series into a DataFrame.

```python
# Creating DataFrame from Pandas Series
series1 = pd.Series([1, 2, 3])
series2 = pd.Series(['A', 'B', 'C'])
df_from_series = pd.DataFrame({'Column1': series1, 'Column2': series2})
print(df_from_series)
```

Each of these methods allows you to create a DataFrame in Pandas using different data sources or combinations. They offer flexibility based on the data structure you have and the way you want to organize it into a DataFrame.

Certainly! Here is a brief explanation of the different ways to create DataFrames in Pandas:

1. Empty DataFrame:
An empty DataFrame is created using the `pd.DataFrame()` constructor with no data provided. It can be used as a starting point to which data can be appended later.

2. DataFrame using Lists:
A DataFrame can be created by passing a list of lists as the data source to `pd.DataFrame()`. The inner lists represent rows, and the outer list encompasses the entire dataset.

3. DataFrame from a Dictionary:
You can create a DataFrame directly from a dictionary. The dictionary keys represent column names, and the corresponding values can be lists or arrays containing the column data.

4. DataFrame from Lists using a Dictionary:
This method involves creating a dictionary with keys as column names and values as lists, and then using `pd.DataFrame()` to construct a DataFrame.

5. DataFrame from Series:
Pandas Series can be used to create a DataFrame. Multiple Series can be combined into a dictionary, and this dictionary can be used to create a DataFrame.

Each of these methods offers flexibility in creating DataFrames based on different data structures and sources, allowing for easy transformation and manipulation of data in a tabular form using Pandas.

2. Describe various methods of handling missing data in python with code segments.

Certainly! Handling missing data is an important aspect of data preprocessing. Pandas provides several methods to deal with missing values in a DataFrame. Here are different approaches to handle missing data with code examples:

1. Identifying Missing Values:

You can identify missing values in a DataFrame using `isnull()` and `notnull()` functions.

```python
import pandas as pd
import numpy as np

# Creating a DataFrame with missing values
data = {
    'A': [1, np.nan, 3, 4],
    'B': [5, 6, np.nan, 8]
}
df = pd.DataFrame(data)

# Check for missing values
print(df.isnull())  # True where values are missing
print(df.notnull())  # True where values are present
```

2. Dropping Missing Values:

The `dropna()` method removes rows or columns with missing values.

```python
# Drop rows with missing values
df_dropna_rows = df.dropna()

# Drop columns with missing values
df_dropna_columns = df.dropna(axis=1)
```

3. Filling Missing Values:

`fillna()` method fills missing values with a specified value.

```python
# Fill missing values with a specific value (e.g., 0)
df_fillna = df.fillna(0)

# Fill missing values with the mean of the column
df_fillna_mean = df.fillna(df.mean())
```

4. Forward Fill and Backward Fill:

Forward fill (`ffill()`) and backward fill (`bfill()`) propagate non-missing values forward or backward.

```python
# Forward fill missing values
df_ffill = df.fillna(method='ffill')

# Backward fill missing values
df_bfill = df.fillna(method='bfill')
```

5. Interpolation:

`interpolate()` estimates missing values based on existing data.

```python
# Linear interpolation for missing values
df_interpolated = df.interpolate()
```

6. Replace Specific Values:

Replace specific values in the DataFrame with new values.

```python
# Replace a specific value with another value (e.g., replace NaN with -1)
df_replace = df.replace(np.nan, -1)
```

7. Handling Missing Data in Specific Columns:

You can handle missing values in specific columns using these methods.

```python
# For a specific column
df['A'].fillna(0, inplace=True)  # Fills missing values in column 'A' with 0
```

```
# Drop rows with missing values in column 'B'
df.dropna(subset=['B'], inplace=True)
```

These methods offer various strategies to handle missing data, allowing for effective data cleaning and preprocessing before analysis or modeling. The choice of method depends on the nature of the data and the objectives of the analysis.

3. Exhibit the outcome of different group-by operations on data frames with python code.

Certainly! Group-by operations in Pandas involve splitting a DataFrame into groups based on some criteria, applying a function to each group, and combining the results. Here are examples showcasing different group-by operations:

1. Grouping and Aggregating Data:

Grouping by a column and applying an aggregation function (e.g., sum, mean) to the groups.

```python
import pandas as pd

# Sample DataFrame
data = {
    'Category': ['A', 'B', 'A', 'B', 'A'],
    'Value': [10, 20, 30, 40, 50]
}
df = pd.DataFrame(data)

# Grouping by 'Category' and computing the sum of 'Value' in each category
grouped_sum = df.groupby('Category')['Value'].sum()
print(grouped_sum)
```

2. Grouping and Multiple Aggregations:

Applying multiple aggregation functions to different columns after grouping.

```python
# Applying multiple aggregation functions after grouping
grouped_multiple = df.groupby('Category').agg({'Value': ['sum', 'mean', 'max']})
print(grouped_multiple)
```

3. Grouping and Transforming Data:

Using `transform()` to apply a function to each group and broadcasting the results back to the original DataFrame.

```python
```

```python
# Grouping and transforming the data
df['Value_mean'] = df.groupby('Category')['Value'].transform('mean')
print(df)
```

4. Grouping and Filtering Data:

Filtering data based on groups using `filter()`.

```python
# Filtering data based on groups
grouped_filter = df.groupby('Category').filter(lambda x: x['Value'].sum() > 60)
print(grouped_filter)
```

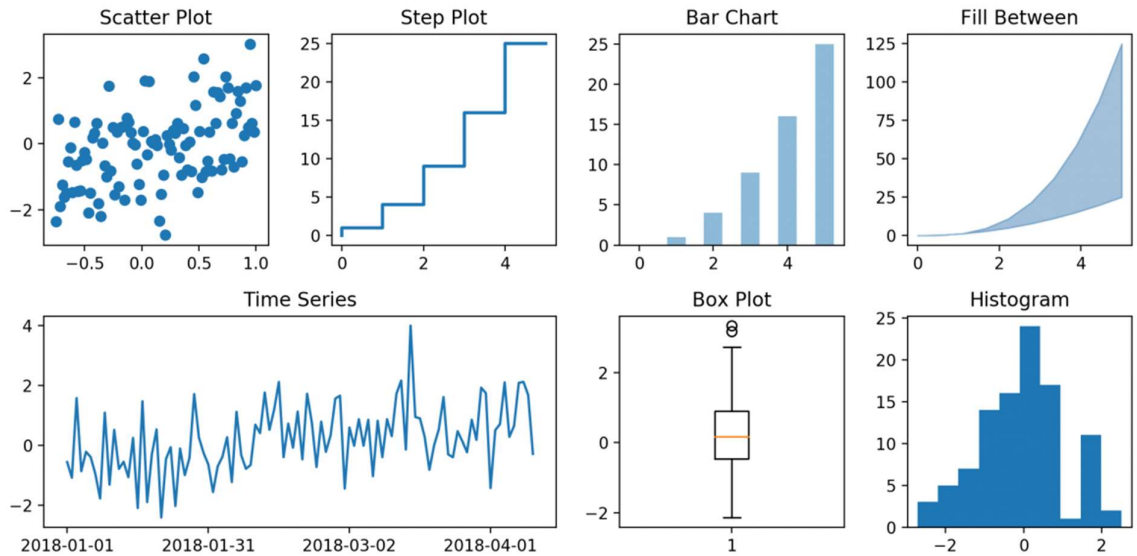
5. Grouping with Multiple Columns:

Grouping by multiple columns and performing aggregation.

```python
# Grouping by multiple columns and performing aggregation
data = {
    'Category': ['A', 'A', 'B', 'B', 'A'],
    'Type': ['X', 'Y', 'X', 'Y', 'X'],
    'Value': [10, 20, 30, 40, 50]
}
df_multiple_columns = pd.DataFrame(data)

grouped_multiple_columns = df_multiple_columns.groupby(['Category', 'Type']).sum()
print(grouped_multiple_columns)
```

These examples illustrate various group-by operations using Pandas, allowing you to split data, perform calculations on groups, and recombine the results, which is a powerful tool for data analysis and manipulation.

UNIT-5



Part B

1. Describe the significance of data modelling and illustrate the important features of a good data visualization.

Significance of Data Modeling:

Data modeling is a crucial aspect of the data analysis process that involves creating mathematical representations or models of real-world systems or phenomena. Its significance lies in several key areas:

1.Understanding Complex Relationships: Data modeling helps in understanding the complex relationships and patterns within the data. It allows identifying correlations, trends, and dependencies between variables.

2.Prediction and Forecasting: Models built using historical data can be used for predictive analysis and forecasting future outcomes. Machine learning and statistical models assist in making predictions and understanding potential scenarios.

3.Decision Making and Optimization:It aids in making informed decisions. Models can simulate scenarios, enabling decision-makers to analyze various options and optimize strategies.

4.Identifying Trends and Patterns: Models reveal hidden patterns and trends that are not immediately obvious in raw data, providing insights for better business strategies.

5.Improving Performance and Efficiency: Models contribute to enhancing efficiency in various processes, be it in production, marketing, resource allocation, or any other aspect of business operations.

Important Features of Good Data Visualization:

Effective data visualization is crucial for interpreting complex information and conveying insights. Key features of a good data visualization include:

1.Clarity and Simplicity: Visualizations should be clear, easily interpretable, and convey the intended message without unnecessary complexity.

2.Relevance: The visualized data should be relevant and targeted to the audience's needs, ensuring it addresses specific questions or goals.

3.Accuracy and Correctness: Accurate representation of data is essential. Visualizations must maintain integrity and avoid misleading representations.

4.Interactivity and Engagement: Interactive features enable users to explore the data and gain deeper insights. Engaging visualizations keep the audience interested.

5.Appropriate Choice of Visualization Type: Choosing the right type of visualization (e.g., bar charts, line graphs, scatter plots, etc.) based on the data and the story you want to tell is crucial for effective communication.

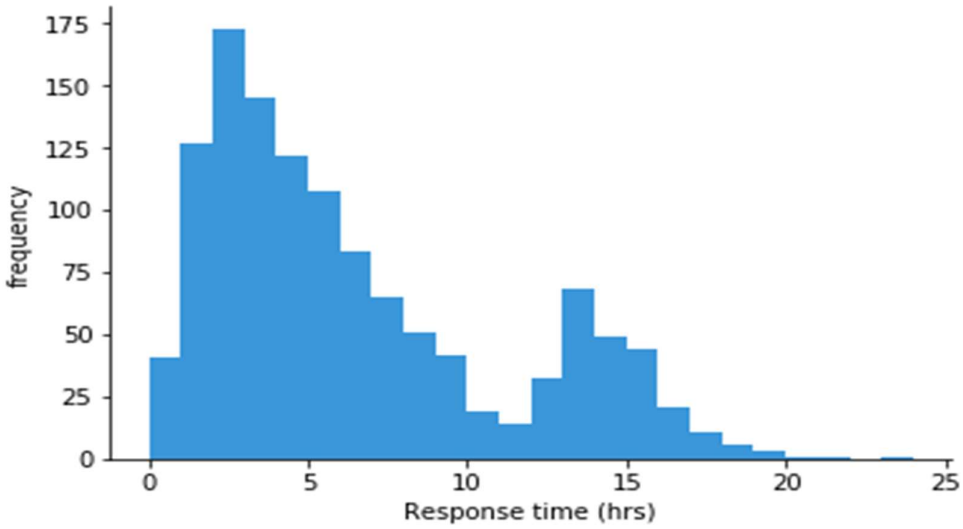
6.Use of Color and Design: Thoughtful use of color, design, and aesthetics improves comprehension and makes visualizations appealing.

7.Consistency and Context: Visualizations should be consistent, and context should be provided, giving a complete picture without misleading the audience.

8. Accessibility: Ensure that visualizations are accessible to all users, considering factors such as color blindness and varying screen sizes.

By incorporating these features, data visualization becomes a powerful tool for effectively communicating insights and facilitating decision-making processes based on data-driven understanding.

2. When analyzing a histogram, what are the features to look for?



When analyzing a histogram, there are several key features and patterns to look for that provide insights into the distribution of the data. Histograms are graphical representations of the distribution of numerical data and can reveal various characteristics. Here are the features to look for and interpret in a histogram:

1. Shape and Symmetry:

- Symmetry: Check if the distribution is symmetrical (bell-shaped), skewed to the right, or skewed to the left.
- Normal Distribution: Look for a bell-shaped curve, which indicates a normal distribution.

2. Central Tendency:

- Mean, Median, Mode: Identify where the peak of the histogram lies in relation to the measures of central tendency. For a normal distribution, the mean, median, and mode are usually close or at the center.

3. Spread and Variability:

- Range: Assess the range covered by the data. A wider range suggests higher variability.
- Standard Deviation: The spread of the histogram could be estimated by examining the width and height of the distribution.

4. Modality:

- Bimodal, Unimodal, Multimodal: Determine if the histogram displays one prominent peak (unimodal), two peaks (bimodal), or multiple peaks (multimodal), indicating different groups or underlying factors in the data.

5. Outliers:

- Identify Outliers: Look for isolated bars or data points that lie far away from the main distribution, indicating potential outliers.

6. Skewness and Kurtosis:

- Skewness: Identify if the distribution is positively skewed (long tail on the right), negatively skewed (long tail on the left), or approximately symmetrical.
- Kurtosis: Assess if the distribution is more peaked (leptokurtic) or flat (platykurtic) compared to a normal distribution.

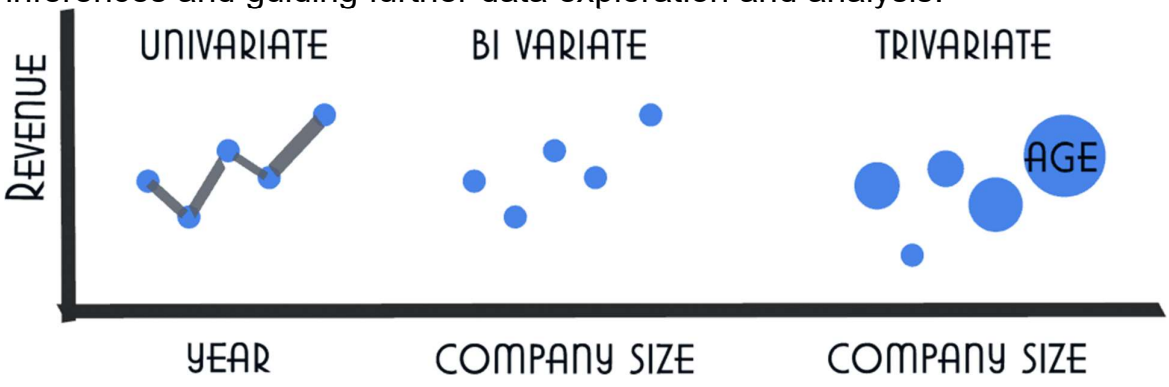
7. Data Patterns and Trends:

- Patterns: Identify patterns or trends within the data, such as periodicity or clusters, which might reveal hidden structures or characteristics of the dataset.

8. Relative Heights of Bars:

- Relative Frequencies: Examine the relative heights of bars to understand the proportion of values within different ranges or bins.

Analyzing these features in a histogram provides valuable insights into the nature and characteristics of the dataset's distribution, aiding in making inferences and guiding further data exploration and analysis.

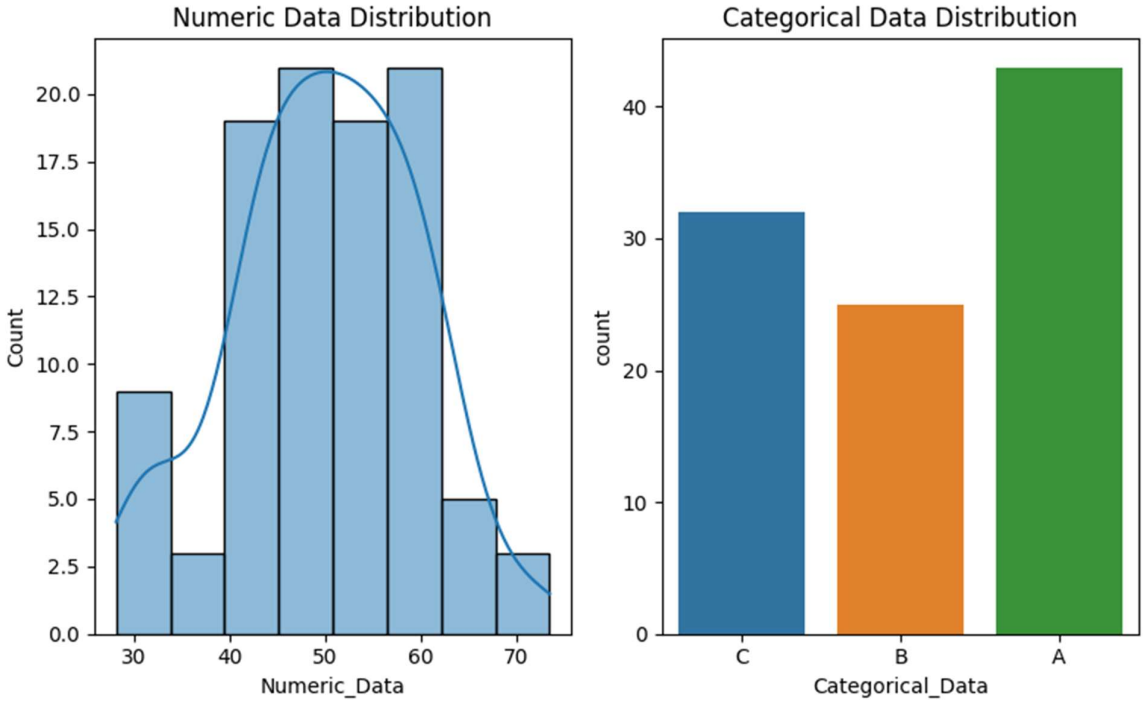


4. Develop python program to plot univariate graphs for numeric and categorical data.

Certainly! Below is an example of a Python program that utilizes Matplotlib and Seaborn libraries to create univariate graphs for both numeric and categorical data:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Generate sample data (replace this with your dataset)
data_numeric = np.random.normal(loc=50, scale=10, size=100)  #
Numeric data
data_categorical = np.random.choice(['A', 'B', 'C'], size=100)  # Categorical
data

# Create numeric data as DataFrame
df_numeric = pd.DataFrame({'Numeric_Data': data_numeric})

# Create categorical data as DataFrame
df_categorical = pd.DataFrame({'Categorical_Data': data_categorical})

# Univariate plot for numeric data
plt.figure(figsize=(8, 5))
plt.subplot(1, 2, 1)
sns.histplot(df_numeric['Numeric_Data'], kde=True)
plt.title('Numeric Data Distribution')

# Univariate plot for categorical data
plt.subplot(1, 2, 2)
sns.countplot(data=df_categorical, x='Categorical_Data')
plt.title('Categorical Data Distribution')

plt.tight_layout()
plt.show()
```

OUTPUT:



This code generates synthetic numeric and categorical data. Replace the sample data creation part with your actual dataset or data loading process. The program uses Seaborn's `histplot` for numeric data and `countplot` for categorical data to create univariate visualizations.
For numeric data, a histogram with a kernel density estimation (KDE) is plotted, showing the distribution and the estimated probability density function. For categorical data, a count plot is used to display the count of each category.

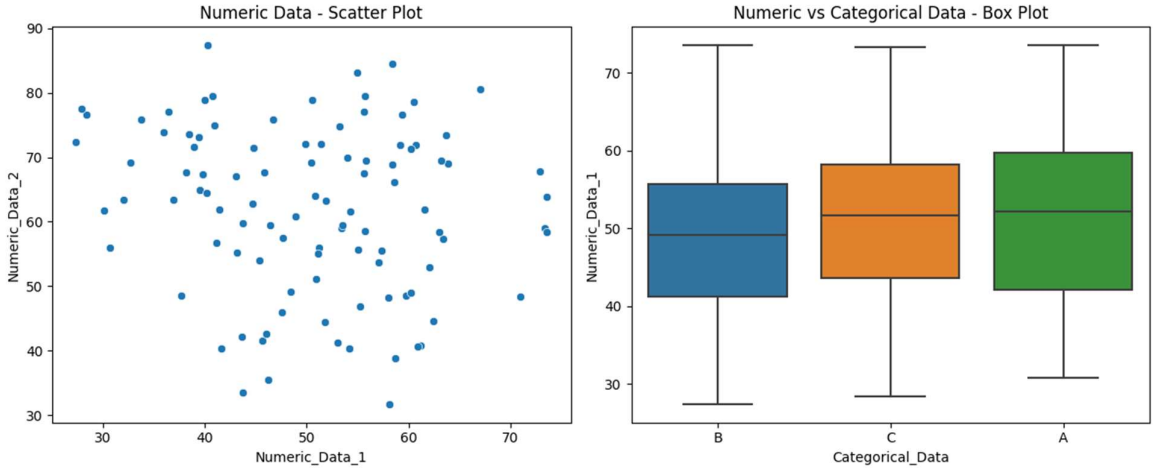
This code will generate a single figure displaying two subplots: one for the numeric data distribution and another for the categorical data distribution. Adjust the size, styling, or other parameters according to your specific requirements.

4. Develop python program to plot Bivariate graphs for numeric and categorical data.

Certainly! Here is an example Python program that utilizes Matplotlib and Seaborn libraries to create bivariate graphs for both numeric and categorical data:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Generate sample data (replace this with your dataset)
data_numeric_1 = np.random.normal(loc=50, scale=10, size=100)  #
Numeric data 1
data_numeric_2 = np.random.normal(loc=60, scale=15, size=100)  #
Numeric data 2
data_categorical = np.random.choice(['A', 'B', 'C'], size=100)  # Categorical
data

# Create DataFrame for numeric data
df_numeric = pd.DataFrame({'Numeric_Data_1': data_numeric_1,
'Numeric_Data_2': data_numeric_2})

# Create DataFrame for categorical data
df_categorical = pd.DataFrame({'Categorical_Data': data_categorical})

# Bivariate plot for numeric data
plt.figure(figsize=(12, 5))

# Scatter plot for two numeric variables
plt.subplot(1, 2, 1)
sns.scatterplot(data=df_numeric, x='Numeric_Data_1',
y='Numeric_Data_2')
plt.title('Numeric Data - Scatter Plot')

# Box plot for numeric and categorical data
plt.subplot(1, 2, 2)
sns.boxplot(data=df_categorical, x='Categorical_Data',
y=df_numeric['Numeric_Data_1'])
plt.title('Numeric vs Categorical Data - Box Plot')

plt.tight_layout()
plt.show()
```

OUTPUT:



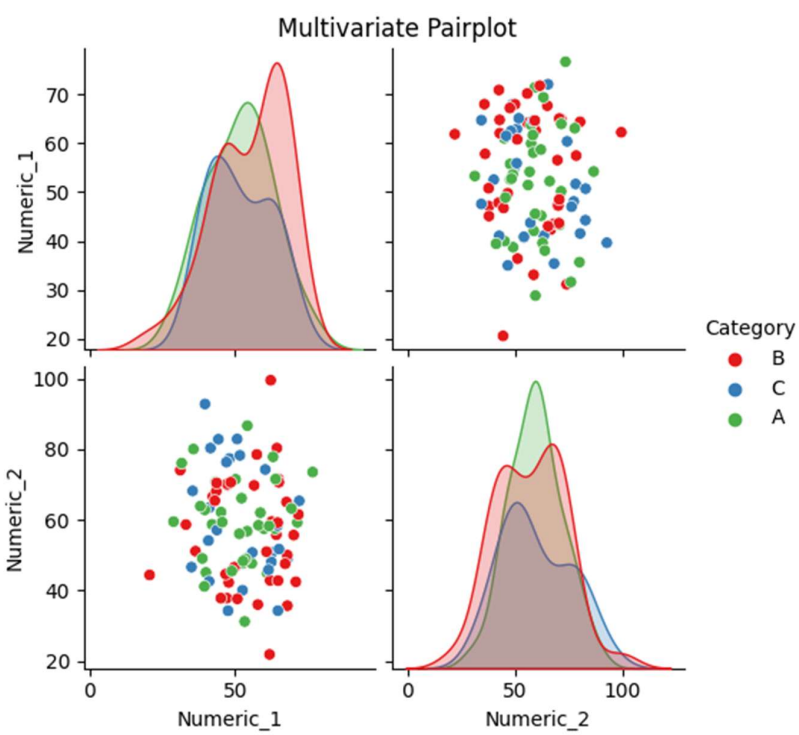
This code generates synthetic numeric and categorical data. Replace the sample data creation part with your actual dataset or data loading process. The program uses Seaborn's `scatterplot` for numeric data to create a scatter plot between two numeric variables and a `boxplot` to compare a numeric variable with a categorical variable.
For numeric data, a scatter plot shows the relationship between two numeric variables. For categorical data, a box plot compares the distribution of a numeric variable across different categories.

This code will generate a single figure displaying two subplots: one for the scatter plot between two numeric variables and another for the box plot representing the relationship between a numeric and a categorical variable. Adjust the size, styling, or other parameters according to your specific requirements.

5. Develop python program to plot Multivariate Graphs

Certainly! Multivariate graphs involve visualizing relationships among multiple variables in a dataset. Below is an example Python program that uses Seaborn to create multivariate graphs:

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

# Generating sample multivariate data (replace this with your dataset)
data = {
    'Numeric_1': np.random.normal(loc=50, scale=10, size=100),
    'Numeric_2': np.random.normal(loc=60, scale=15, size=100),
    'Category': np.random.choice(['A', 'B', 'C'], size=100),
    'Group': np.random.choice(['X', 'Y'], size=100)
}

# Create DataFrame
df = pd.DataFrame(data)

# Multivariate plot - Pairplot
sns.pairplot(df, hue='Category', kind='scatter', palette='Set1')
plt.suptitle('Multivariate Pairplot', y=1.02)
plt.show()
```



This code generates synthetic multivariate data. Replace the sample data creation part with your actual dataset or data loading process. The program uses Seaborn's `pairplot()` to create a grid of pairwise relationships among the variables, allowing for the visualization of relationships between numeric variables and categorical variables.

The `pairplot` displays scatterplots for numeric variables and histograms for the diagonal, while different colors represent different categories in the dataset.

Adjust the palette, style, and other parameters in `pairplot()` according to your specific requirements for data visualization.

6. Compare the features of MAtplotlib and seaborn

Certainly! Matplotlib and Seaborn are both widely used Python libraries for data visualization, but they have distinct features and are used for different purposes. Here's a comparison of their features:

Matplotlib:

1. Base Library: Matplotlib is the fundamental plotting library in Python and offers a wide range of functionalities for creating static, interactive, and publication-quality visualizations.

2. Customizability: Highly customizable and allows low-level control over every aspect of the plot. Users have fine-grained control over details like line styles, markers, colors, text, etc.

3. Wide Range of Plots: Capable of creating a broad range of plots, from basic line plots, scatter plots, bar plots to complex visualizations.

4. Compatible with Pandas: Matplotlib seamlessly integrates with Pandas for visualizing DataFrames.

5. Explicitness: Produces less aesthetically pleasing plots by default, but with more manual adjustments, can produce professional-looking visualizations.

Seaborn:

1. Statistical Data Visualization: Seaborn is built on top of Matplotlib and provides a high-level interface for creating attractive and informative statistical graphics.

2. Default Aesthetics: Provides pleasant and visually appealing default aesthetics, which is beneficial for quick and beautiful visualization of data.

3. Statistical Plotting: Offers specialized support for statistical plots like violin plots, box plots, pair plots, etc., which are simpler to create in Seaborn compared to Matplotlib.

4. Ease of Use: Seaborn abstracts complex functions and provides an easier syntax, making it suitable for rapid data exploration.

5. Grids and Color Palettes: Seaborn has grid-based layouts and a variety of color palettes built-in, simplifying the creation of complex multi-plot grids.

Summary:

- Matplotlib is a powerful, flexible library providing precise control over individual plot elements, but may require more effort for creating attractive visualizations.
- Seaborn offers a higher-level interface and default aesthetics to quickly create appealing statistical graphics, making it suitable for exploratory data analysis and visualization.

For simple, quick, and beautiful statistical visualizations, Seaborn might be more convenient. However, for more customized and intricate plots, Matplotlib is highly adaptable and provides fine-grained control over visual elements. Often, users utilize both libraries: Seaborn for quick data exploration and Matplotlib for more specialized or customized visualizations.

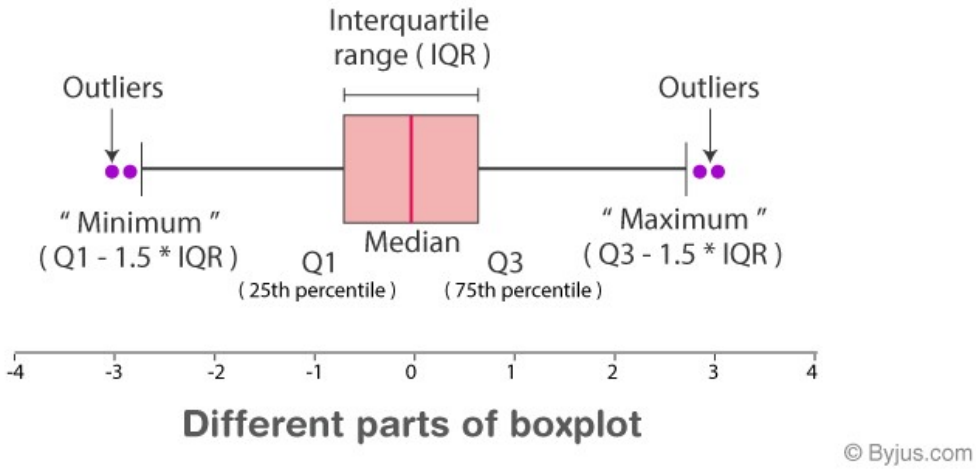
Features	Matplotlib	Seaborn
Functionality	It is utilized for making basic graphs. Datasets are visualised with the help of bargraphs, histograms, piecharts, scatter plots, lines and so on.	Seaborn contains a number of patterns and plots for data visualization. It uses fascinating themes. It helps in compiling whole data into a single plot. It also provides distribution of data.
Syntax	It uses comparatively complex and lengthy syntax. Example: Syntax for bargraph- matplotlib.pyplot.bar(x_axis, y_axis).	It uses comparatively simple syntax which is easier to learn and understand. Example: Syntax for bargraph- seaborn.barplot(x_axis

		, y_axis).
Dealing Multiple Figures	We can open and use multiple figures simultaneously. However they are closed distinctly. Syntax to close one figure at a time: matplotlib.pyplot.close(). Syntax to close all the figures: matplotlib.pyplot.close("all")	Seaborn sets time for the creation of each figure. However, it may lead to (OOM) out of memory issues
Visualization	Matplotlib is well connected with Numpy and Pandas and acts as a graphics package for data visualization in python. Pyplot provides similar features and syntax as in MATLAB. Therefore, MATLAB users can easily study it.	Seaborn is more comfortable in handling Pandas data frames. It uses basic sets of methods to provide beautiful graphics in python.
Pliability	Matplotlib is a highly customized and robust	Seaborn avoids overlapping of plots with the help of its

		default themes
Data Frames and Arrays	Matplotlib works efficiently with data frames and arrays.It treats figures and axes as objects. It contains various stateful APIs for plotting. Therefore plot() like methods can work without parameters.	Seaborn is much more functional and organized than Matplotlib and treats the whole dataset as a single unit. Seaborn is not so stateful and therefore, parameters are required while calling methods like plot()
Use Cases	Matplotlib plots various graphs using Pandas and Numpy	Seaborn is the extended version of Matplotlib which uses Matplotlib along with Numpy and Pandas for plotting graphs

PART C

1. **What type of data is box-plots usually used for? Why? What information could you gain from a box-plot?**



Boxplots are particularly useful for visualizing the distribution and statistical summary of numerical data. They are commonly used for the following reasons:

Type of Data Suitable for Boxplots:

Boxplots are primarily used for displaying the distribution of continuous numerical data. They are especially effective for:

1. Univariate Analysis: Analyzing the distribution of a single variable.
2. Comparing Multiple Groups: Comparing the distribution of a numerical variable across different groups or categories.

Information Gained from a Boxplot:

A boxplot provides the following key information about a dataset:

1. Center and Spread of Data:
   - Median (Q2):The line inside the box represents the median value of the dataset.
   - Interquartile Range (IQR): The box's length represents the IQR, showing the spread of the middle 50% of the data. It extends from the first quartile (Q1) to the third quartile (Q3).

2. Variability and Spread of the Data:
   - Whiskers: The whiskers extend to 1.5 times the IQR from the edges of the box, showing the range of the dataset. They represent the bulk of the data, excluding outliers.

- Outliers: Data points beyond the whiskers are marked as individual points, giving insights into potential outliers.

3. Skewness and Symmetry:
   - Symmetry or Skewness: The distribution's symmetry can be inferred by comparing the lengths of the box on each side of the median line. Skewed distributions have unequal box lengths.

4. Presence of Extremes:
   -Identifying Extremes: Outliers can be identified, indicating potential extreme values in the dataset.

5. Comparison between Groups:
   - Comparing Groups: When multiple boxplots are plotted together, they allow easy visual comparison of the central tendency and spread of data between different groups.

Interpretation:

- Center: The median is the central tendency of the data.
- Spread: The width of the box and whiskers indicates the spread or dispersion of the data.
- Outliers: Points beyond the whiskers help identify potential anomalies or extreme values.
- Comparison: It enables visual comparison of the central tendency and variability between groups.

Boxplots are effective for providing a concise summary of the distribution of the data, making it easier to identify outliers, understand the spread, and compare groups, which are crucial for exploratory data analysis and understanding the characteristics of a dataset.

2. When will you use a histogram and when will you use a bar chart? Explain with an example.

Histograms and bar charts are both types of graphical representations, but they serve different purposes and are used for different types of data.

Histogram:

- Use: Histograms are used to display the distribution of continuous numerical data.
- X-axis: Represents the range of values grouped into intervals or bins.
- Y-axis: Represents the frequency or count of data points within each interval.

Bar Chart:

- Use: Bar charts are used to display and compare categorical data or discrete values.
- X-axis: Represents different categories or groups.
- Y-axis: Represents the frequency, count, or any numerical value associated with each category.

Example Differentiating Histogram and Bar Chart:

Consider a dataset containing students' test scores:

Example 1: Histogram

If you want to visualize the distribution of these test scores to understand how scores are distributed across different ranges, you would use a histogram. For instance, consider the following test scores:

```
import matplotlib.pyplot as plt
import numpy as np

# Example test scores (continuous numerical data)
test_scores = np.random.normal(70, 10, 100)  # Generating 100 test
scores

plt.hist(test_scores, bins=10, edgecolor='black')
plt.title('Distribution of Test Scores')
plt.xlabel('Scores')
plt.ylabel('Frequency')
plt.show()
```

This code generates a histogram displaying the distribution of test scores in intervals (bins) and shows how many students fall within each score range.

Example 2: Bar Chart

If you want to compare the average test scores between different classes or subjects, a bar chart is suitable. For instance, considering the average scores of students in different subjects:

```
# Example average scores for different subjects (categorical data)
subjects = ['Math', 'Science', 'History', 'English']
average_scores = [80, 75, 70, 85]

plt.bar(subjects, average_scores, color='skyblue')
plt.title('Average Scores by Subject')
plt.xlabel('Subjects')
plt.ylabel('Average Score')
plt.show()
```

This code generates a bar chart displaying the average scores in different subjects, making it easy to compare the performance across different categories (subjects, in this case).

In summary, use a histogram to understand the distribution of continuous numerical data and a bar chart to compare categorical or discrete data among different categories or groups.

3. For what type of data is scatter plot usually used for? Identify the features visible in scatterplots.

Scatter plots are typically used to visualize the relationship or association between two continuous numerical variables. They are particularly useful for identifying patterns, trends, clusters, or correlations between variables.

Features Visible in Scatterplots:

1. Relationship Between Variables:
   - Positive Correlation: Points tend to rise as you move from left to right on the plot.
   - Negative Correlation: Points tend to fall as you move from left to right on the plot.
   - No Correlation: Points show no discernible pattern or trend.

2. Outliers:
   - Outlying points that deviate significantly from the general pattern can be easily identified.

3. Clustering and Groups:
   - Clusters or groups of points can indicate subsets within the data, suggesting potential subgroups or distinct relationships.

4. Strength of Relationship (Correlation):
   - The closeness of points to a linear trendline (if fitted) or to each other indicates the strength of the relationship between variables.

5. Direction and Shape:
   - The direction of the relationship (positive or negative) is indicated by the general trend of the points.
   - Non-linear relationships can also be observed, showing curved or other patterns in the scatter.

6. Scatterplot Matrix:
   - When analyzing multiple variables, a scatterplot matrix visualizes the relationships between all pairs of variables, revealing patterns across the entire dataset.

7. Identification of Trends:
   - A line or curve of best fit can be added to visualize the general trend between variables.

Example Scenario:

For instance, consider a dataset comparing the relationship between hours spent studying and exam scores for a group of students. A scatter

plot of this data might show how exam scores change based on the number of study hours. Here's an example:

```
import matplotlib.pyplot as plt
import numpy as np

# Example dataset: Hours studied and corresponding exam scores
hours_studied = np.array([2, 3, 4, 5, 6, 7, 8, 9])
exam_scores = np.array([60, 65, 70, 72, 80, 85, 88, 90])

plt.scatter(hours_studied, exam_scores, color='blue')
plt.title('Relationship between Study Hours and Exam Scores')
plt.xlabel('Hours Studied')
plt.ylabel('Exam Scores')
plt.show()
```

This code generates a scatter plot displaying the relationship between hours studied (independent variable) and exam scores (dependent variable). It illustrates how the two variables may relate to each other, whether they have a positive, negative, or no evident relationship.

Scatter plots are essential for identifying patterns, relationships, and outliers in datasets with two continuous variables, making them a powerful tool for exploratory data analysis and initial understanding of data relationships.